

Path Planning

路径规划

WPILib Docs 2026

中英对照 · Java

[引言](#) → [Trajectory Tutorial](#) → [PathWeaver](#) → [Choreo](#)

15 篇文档 · [按 WPILib 目录顺序汇编](#)

Contents / 目录

Path Planning / 路径规划	3
Trajectory Tutorial / 轨迹教程	5
Trajectory Tutorial Overview / 轨迹教程概览	6
Step 1: Characterizing Your Robot Drive / 步骤 1: 对机器人底盘进行表征	9
Step 2: Entering the Calculated Constants / 步骤 2: 输入计算得到的常数	16
Step 3: Creating a Drive Subsystem / 步骤 3: 创建底盘 subsystem	18
Step 4: Creating and Following a Trajectory / 步骤 4: 创建并跟随轨迹	25
PathWeaver / PathWeaver	30
Introduction to PathWeaver / PathWeaver 简介	31
Creating a Pathweaver Project / 创建 PathWeaver 项目	32
Visualizing PathWeaver Trajectories / 可视化 PathWeaver 轨迹	36
Creating Autonomous Routines / 创建自主例程	40
Importing a PathWeaver JSON / 导入 PathWeaver JSON	41
Adding field images to PathWeaver / 向 PathWeaver 添加场地图像	43
Choreo / Choreo	45

Path Planning / 路径规划

Path Planning is the process of creating and following trajectories. These paths use the WPILib trajectory APIs for generation and a [LTV Unicycle Controller](#) for following. This section highlights the process of characterizing your robot for system identification, trajectory following, and usage of PathWeaver. Users may also want to read the [generic trajectory following documents](#) for additional information about the API and non-commandbased usage.

路径规划是创建并跟随轨迹的过程。这些路径使用 WPILib 轨迹 API 生成，并使用 [LTV 独轮车控制器](#) 跟随。本节重点介绍如何为机器人进行 SysId 表征、跟随轨迹，以及使用 PathWeaver。若要了解 API 和不使用 Command-Based 框架的用法，还可以阅读[通用轨迹跟随文档](#)。

Notice on Swerve Support / 关于 swerve 支持的说明

Swerve support in path following has a couple of limitations that teams need to be aware of:

使用 swerve 跟随路径存在一些限制，队伍需要注意：

- WPILib currently does not support swerve in simulation, please see [this](#) pull request.
- WPILib 目前不支持在仿真中使用 swerve，请参阅[此](#)拉取请求。
- Pathweaver and Trajectory following currently do not incorporate independent heading. Path following using the WPILib trajectory framework on swerve will be the same as a DifferentialDrive robot. Both [Choreo](#) and [PathPlanner](#) have swerve support.
- PathWeaver 和轨迹跟随目前不支持独立朝向。使用 WPILib 轨迹框架进行 swerve 路径跟随，与差速驱动机器人相同。[Choreo](#) 和 [PathPlanner](#) 都支持 swerve。

We are sorry for the inconvenience.

对于由此造成的不便，我们深表歉意。

- [Trajectory Tutorial](#)
- [轨迹教程](#)
 - [Trajectory Tutorial Overview](#)
 - [轨迹教程概览](#)
 - [Step 1: Characterizing Your Robot Drive](#)
 - [步骤 1: 对机器人底盘进行表征](#)
 - [Step 2: Entering the Calculated Constants](#)
 - [步骤 2: 输入计算得到的常数](#)
 - [Step 3: Creating a Drive Subsystem](#)
 - [步骤 3: 创建底盘 subsystem](#)
 - [Step 4: Creating and Following a Trajectory](#)
 - [步骤 4: 创建并跟随轨迹](#)
- [PathWeaver](#)
- [PathWeaver](#)
 - [Introduction to PathWeaver](#)
 - [PathWeaver 简介](#)

- [Creating a Pathweaver Project](#)
- [创建 PathWeaver 项目](#)
- [Visualizing PathWeaver Trajectories](#)
- [可视化 PathWeaver 轨迹](#)
- [Creating Autonomous Routines](#)
- [创建自主例程](#)
- [Importing a PathWeaver JSON](#)
- [导入 PathWeaver JSON](#)
- [Adding field images to PathWeaver](#)
- [向 PathWeaver 添加场地图像](#)
- [Choreo](#)
- [Choreo](#)

Trajectory Tutorial / 轨迹教程

This is full tutorial for implementing trajectory generation and following on a differential-drive robot. The full code used in this tutorial can be found in the [RamseteCommand](#) example project ([Java](#)).

这是一份在差速驱动机器人上实现轨迹生成与跟随的完整教程。教程中使用的完整代码可在 [RamseteCommand](#) 示例项目（[Java](#)）中找到。

- [Trajectory Tutorial Overview](#)
- 轨迹教程概览
- [Step 1: Characterizing Your Robot Drive](#)
- 步骤 1: 对机器人底盘进行表征
- [Step 2: Entering the Calculated Constants](#)
- 步骤 2: 输入计算得到的常数
- [Step 3: Creating a Drive Subsystem](#)
- 步骤 3: 创建底盘 subsystem
- [Step 4: Creating and Following a Trajectory](#)
- 步骤 4: 创建并跟随轨迹

Trajectory Tutorial Overview / 轨迹教程概览

Warning / 警告

This tutorial uses the Ramsete Controller which has been [deprecated](#). The replacement is [LTV Unicycle Controller](#) which has more intuitive tuning. We are looking for volunteers to [update this tutorial](#).

本教程使用已被弃用的 Ramsete 控制器。替代方案是调参更直观的 [LTV 独轮车控制器](#)。我们正在招募志愿者更新本教程。

Note / 说明

Before following this tutorial, it is helpful (but not strictly necessary) to have a baseline familiarity with WPILib's [PID control](#), [feedforward](#), and [trajectory](#) features.

在学习本教程前，最好先对 WPILib 的 [PID 控制](#)、[前馈](#)和[轨迹](#)功能有基本了解，但这并非严格要求。

Note / 说明

The robot code in this tutorial uses the [command-based](#) framework. The command-based framework is strongly recommended for beginning and intermediate teams.

本教程中的机器人代码使用 [Command-Based](#) 框架。强烈建议初级和中级队伍使用 [Command-Based](#) 框架。

The goal of this tutorial is to provide “end-to-end” instruction on implementing a trajectory-following autonomous routine for a differential-drive robot. By following this tutorial, readers will learn how to:

本教程旨在完整介绍如何为差速驱动机器人实现轨迹跟随自主例程。通过学习本教程，读者将学会：

1. Accurately characterize their robot's drivetrain to obtain accurate feedforward calculations and approximate feedback gains.
1. 准确表征机器人底盘，以获得准确的前馈计算结果和近似的反馈增益。
2. Configure a drive subsystem to track the robot's pose using WPILib's odometry library.
2. 配置底盘 subsystem，使用 WPILib 的 odometry 库跟踪机器人的 pose。
3. Generate a simple trajectory through a set of waypoints using WPILib's TrajectoryGenerator class.
3. 使用 WPILib 的 TrajectoryGenerator 类，生成经过一组路径点的简单轨迹。
4. Follow the generated trajectory in an autonomous routine using WPILib's RamseteCommand class with the calculated feedforward/feedback gains and pose.
4. 使用计算得到的前馈、反馈增益和 pose，通过 WPILib 的 RamseteCommand 类在自主例程中跟随生成的轨迹。

This tutorial is intended to be approachable for teams without a great deal of programming expertise. While the WPILib library offers significant flexibility in the manner in which its trajectory-following features are implemented, closely following the implementation outlined in this tutorial should provide teams with a relatively-simple, clean, and repeatable solution for autonomous movement.

本教程力求让编程经验不多的队伍也能上手。尽管 WPILib 库在轨迹跟随功能的实现方式上提供了很大灵活性，紧密遵循本教程所述实现，仍应能为队伍提供一种相对简单、清晰且可重复的自主运动方案。

The full robot code for this tutorial can be found in the [RamseteCommand Example Project \(Java\)](#).

本教程的完整机器人代码可在 [RamseteCommand 示例项目 \(Java\)](#) 中找到。

Why Trajectory Following? / 为什么要跟随轨迹?

FRC® games often feature autonomous tasks that require a robot to effectively and accurately move from a known starting location to a known scoring location. Historically, the most common solution for this sort of task in FRC has been a “drive-turn-drive” approach – that is, drive forward by a known distance, turn by a known angle, and drive forward by another known distance.

FRC® 比赛常包含自主任务，需要机器人有效且准确地从已知起点移动到已知得分位置。过去，在 FRC 中解决这类任务最常见的方式是“直行—转向—直行”：先向前行驶已知距离，转过已知角度，再向前行驶另一段已知距离。

While the “drive-turn-drive” approach is certainly functional, in recent years teams have begun tracking smooth trajectories which require the robot to drive and turn at the same time. While this is a fundamentally more-complicated technical task, it offers significant benefits: in particular, since the robot no longer has to stop to change directions, the paths can be driven much faster, allowing a robot to score more game pieces during the autonomous period.

“直行—转向—直行”当然可行，但近年来，队伍开始跟随平滑轨迹，让机器人同时行驶和转向。虽然这在技术上更加复杂，却有显著优点：尤其是机器人不再需要停下来改变方向，因此能够更快地行驶路径，在自主阶段取得更多场上物件的得分。

Beginning in 2020, WPILib now supplies teams with working, advanced code solutions for trajectory generation and tracking, significantly lowering the “barrier-to-entry” for this kind of advanced and effective autonomous motion.

从 2020 年起，WPILib 为队伍提供可用的高级轨迹生成和跟踪代码方案，大幅降低了实现这种高效自主运动的门槛。

Required Equipment / 所需设备

To follow this tutorial, you will need ready access to the following materials:

学习本教程需要能够使用以下材料：

1. A differential-drive robot (such as the [AndyMark AM14U5](#)), equipped with:
 1. 一台差速驱动机器人（例如 [AndyMark AM14U5](#)），并配备：
 - Quadrature encoders for measuring the wheel rotation of each side of the drive.
 - 测量底盘两侧车轮旋转的正交编码器。
 - A gyroscope for measuring robot heading.
 - 测量机器人朝向的陀螺仪。
 1. A driver-station computer configured with:
 1. 一台配置了以下软件的 Driver Station 计算机：
 - [FRC Driver Station](#).
 - [FRC Driver Station](#).
 - [WPILib](#).
 - [WPILib](#).

- [The System Identification Toolsuite.](#)
- SysId 工具集。

Step 1: Characterizing Your Robot Drive / 步骤 1：对机器人底盘进行表征

Note / 说明

For detailed instructions on using the System Identification tool, see its [dedicated documentation](#).

有关使用 SysId 工具的详细说明，请参阅其[专门文档](#)。

Note / 说明

The drive identification process requires ample space for the robot to drive. Be sure to have at least a 10' stretch (ideally closer to 20') in which the robot can drive during the identification routine.

底盘 SysId 过程需要足够的机器人行驶空间。确保在运行 SysId 例程时，机器人有至少 10 英尺（最好接近 20 英尺）的直线行驶距离。

Note / 说明

The identification data for this tutorial has been generously provided by Team 5190, who generated it as part of a demonstration of this functionality at the 2019 North Carolina State University P2P Workshop.

本教程的表征数据由 5190 队慷慨提供。他们在 2019 年北卡罗来纳州立大学 P2P 研讨会上演示此功能时生成了这些数据。

Before accurately following a path with a robot, it is important to have an accurate model for how the robot moves in response to its control inputs. Determining such a model is a process called “system identification.” WPILib’s System Identification tool can accurately determine such a model.

要让机器人准确跟随路径，首先需要建立一个准确的模型，描述机器人如何响应控制输入而运动。确定这种模型的过程称为“SysId”。WPILib 的 SysId 工具能够准确确定这种模型。

Gathering the Data / 采集数据

We begin by gathering our drive identification data.

首先采集底盘的 SysId 数据。

1. [Configure and Deploy your robot project](#).
1. 配置并部署机器人项目。
2. [Run the identification Routine](#).
2. 运行 SysId 例程。

Analyzing the Data / 分析数据

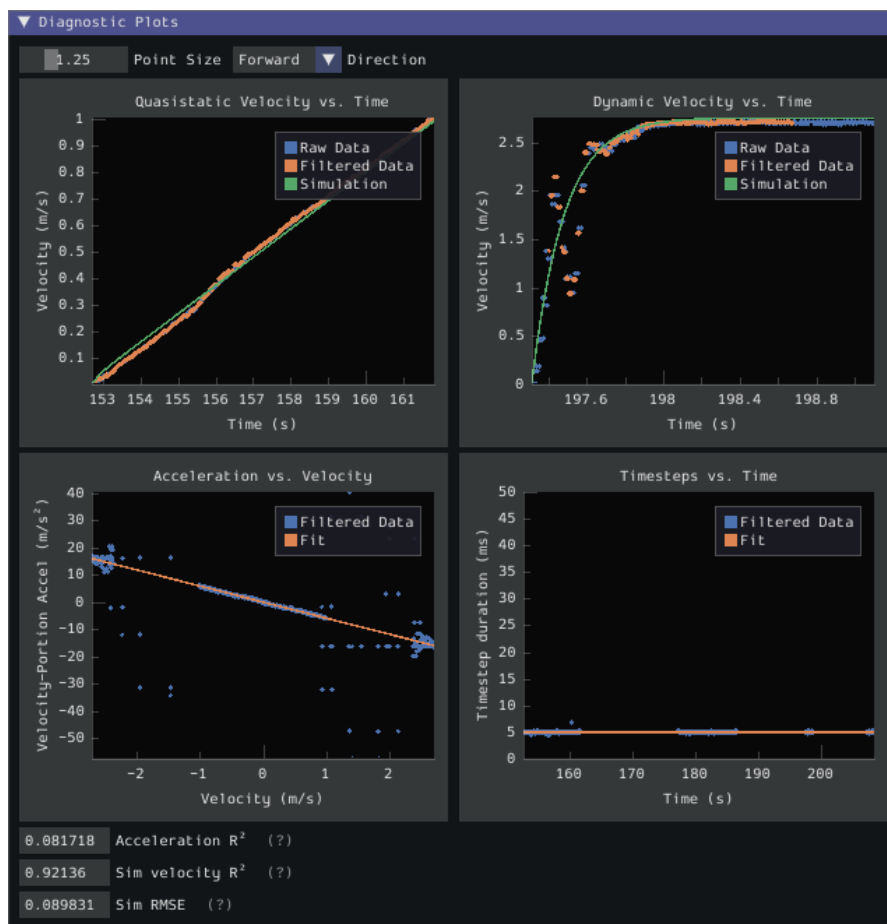
Once the identification routine has been run and the data file has been saved, it is time to [open it in the analysis pane](#).

运行 SysId 例程并保存数据文件后，需要在分析窗格中打开该文件。

Checking Diagnostics / 检查诊断结果

Per the [system identification guide](#), we first view the diagnostics to ensure that our data look reasonable:

按照 SysId 指南，我们首先查看诊断结果，确认数据看起来合理：



SysId diagnostics panel. / SysId 诊断面板。

As our data look reasonably linear, and the fit metrics are within acceptable parameters, we proceed to the next step.

由于数据看起来相当线性，拟合指标也在可接受范围内，我们继续下一步。

Record Feedforward Gains / 记录前馈增益

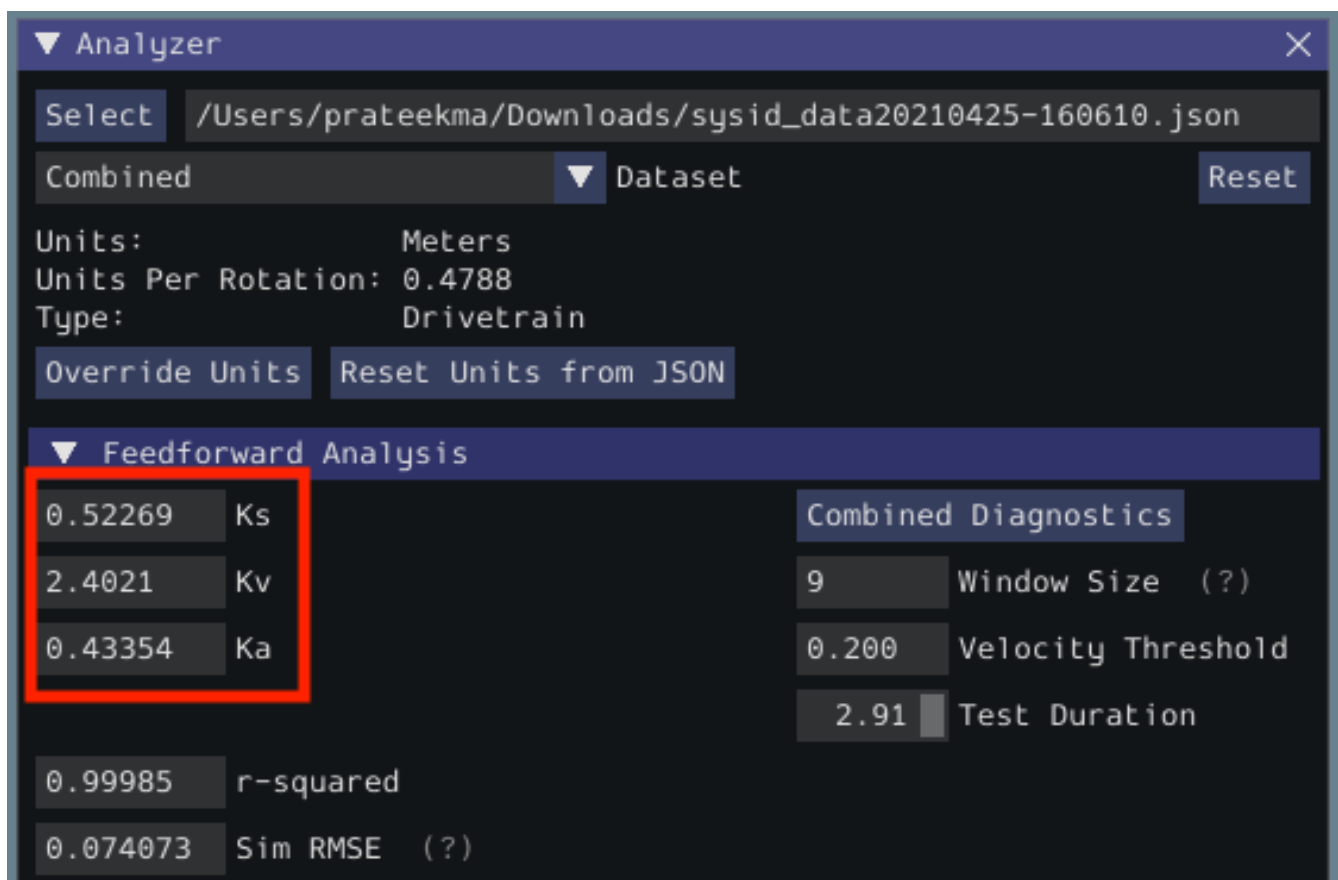
Note / 说明

Feedforward gains do not, in general, transfer across robots. Do not use the gains from this tutorial for your own robot.

通常，不同机器人之间的前馈增益不能直接通用。不要把本教程中的增益用于自己的机器人。

We now record the feedforward gains calculated by the tool:

现在记录工具计算出的前馈增益：



Highlights where to get the kS, kV, kA, and r-squared results from in the center of the window. / 突出显示窗口中部获取 kS、kV、kA 和 r-squared 结果的位置。

Since our wheel diameter was specified in meters, our feedforward gains are in the following units:

由于轮径以米指定，前馈增益采用以下单位：

- kS: Volts
- kS: 伏特
- kV: Volts * Seconds / Meters
- kV: 伏特 * 秒 / 米
- kA: Volts * Seconds² / Meters
- kA: 伏特 * 秒² / 米

If you have specified your units correctly, your feedforward gains will likely be within an order of magnitude of the ones reported here (a possible exception exists for k_A , which may be vanishingly small if your robot is light). If they are not, it is possible you specified one of your drive parameters incorrectly when generating your robot project. A good test for this is to calculate the “theoretical” value of k_V , which is 12 volts divided by the theoretical free speed of your drivetrain (which is, in turn, the free speed of the motor times the wheel circumference divided by the gear reduction). This value should agree very closely with the k_V measured by the tool – if it does not, you have likely made an error somewhere.

如果单位设置正确，前馈增益很可能与此处报告的数值处于同一个数量级（ k_A 可能例外：如果机器人很轻，它可能小到几乎为零）。如果不是，生成机器人项目时可能错误地设置了某个底盘参数。一个很好的检验方法是计算 k_V 的“理论”值：12 伏特除以底盘的理论空载速度，而后者等于电机空载速度乘以车轮周长，再除以减速比。这个值应与工具测得的 k_V 非常接近；如果不接近，很可能某处出了错。

Calculate Feedback Gains / 计算反馈增益

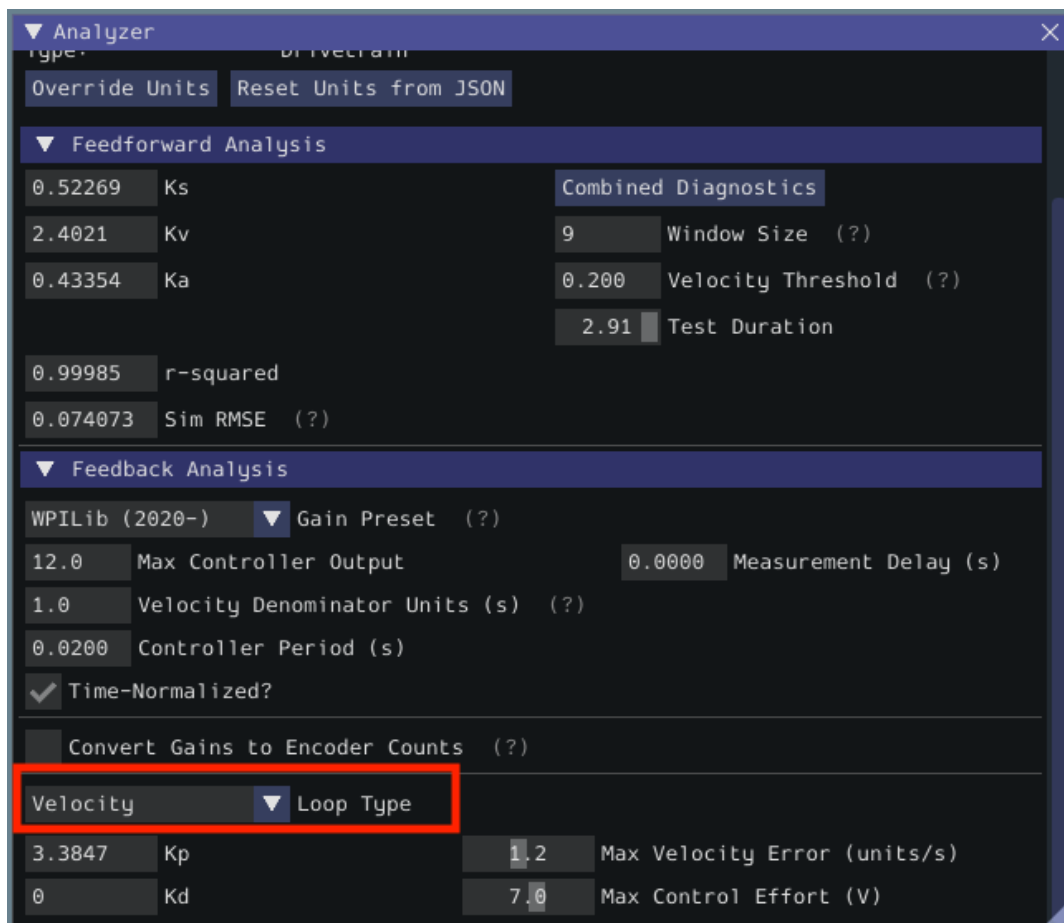
Note / 说明

Feedback gains do not, in general, transfer across robots. Do not use the gains from this tutorial for your own robot.

通常，不同机器人之间的反馈增益不能直接通用。不要把本教程中的增益用于自己的机器人。

We now [calculate the feedback gains](#) for the PID control that we will use to follow the path. Trajectory following with WPILib’s RAMSETE controller uses velocity closed-loop control, so we first select Velocity mode in the identification tool:

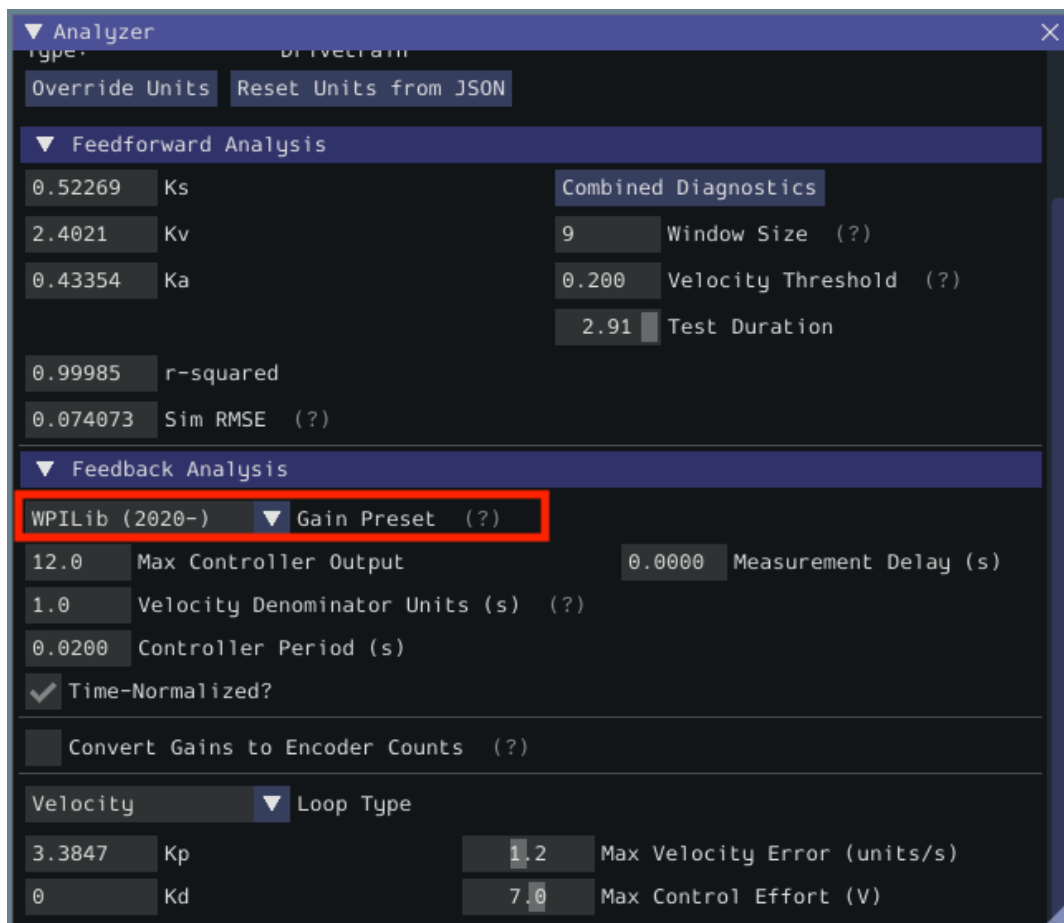
现在[计算反馈增益](#)，用于路径跟随时 PID 控制。使用 WPILib 的 RAMSETE 控制器跟随轨迹，需要速度闭环控制，因此首先在 SysId 工具中选择 Velocity 模式：



How to specify velocity mode in the "Loop Type" dropdown. / 展示如何在“Loop Type”下拉菜单中指定速度模式。

Since we will be using the WPILib PIDController for our velocity loop, we furthermore select the WPILib (2020-) option from the drop-down “presets” menu. This is very important, as the feedback gains will not be in the correct units if we do not select the correct preset:

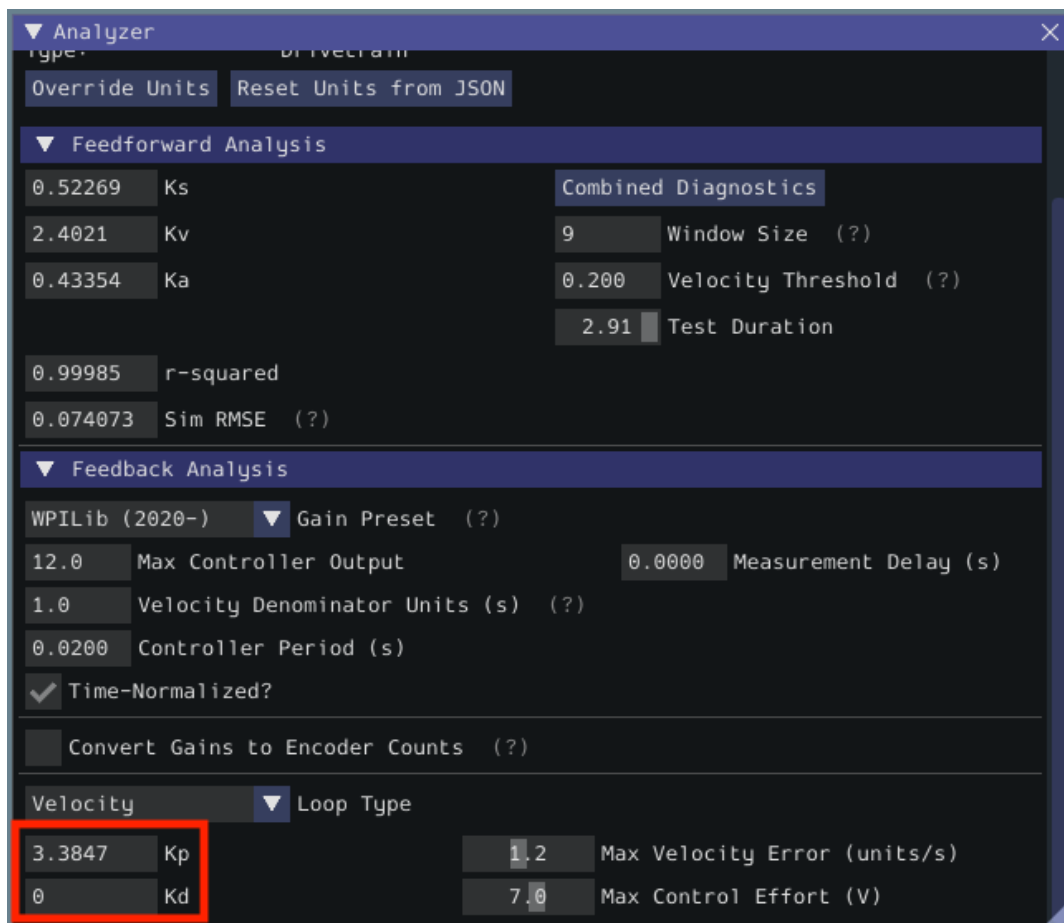
由于速度回路使用 WPILib PIDController，我们还需在“presets”下拉菜单中选择 WPILib (2020-) 选项。这非常重要：如果未选择正确预设，反馈增益的单位就会不正确：



Highlights choosing "WPILib (2020-)" from the "Gain Setting Preset" dropdown. / 突出显示从 "Gain Setting Preset" 下拉菜单选择 "WPILib (2020-)" 的操作。

Finally, we calculate and record the feedback gains for our control loop. Since it is a velocity controller, only a P gain is required:

最后，计算并记录控制回路的反馈增益。因为它是速度控制器，只需要 P 增益：



Highlights the calculated kP and kD controller gains. / 突出显示计算出的 kP 和 kD 控制器增益。

Assuming we have done everything correctly, our proportional gain will be in units of Volts * Seconds / Meters. Thus, our calculated gain means that, for each meter per second of velocity error, the controller will output an additional 3.38 volts.

假设所有操作都正确，比例增益的单位应为伏特 * 秒 / 米。因此，计算出的增益意味着速度误差每增加一米每秒，控制器就会额外输出 3.38 伏特。

Step 2: Entering the Calculated Constants / 步骤 2: 输入计算得到的常数

Now that we have our system constants, it is time to place them in our code. The recommended place for this is the Constants file of the [standard command-based project structure](#).

现在我们已经得到了系统常数，可以把它们写入代码。推荐的位置是[标准 Command-Based 项目结构](#)中的 Constants 文件。

The relevant parts of the constants file from the RamseteCommand Example Project ([Java](#)) can be seen below.

下方展示 RamseteCommand 示例项目 ([Java](#)) 常数文件的相关部分。

Feedforward/Feedback Gains / 前馈 / 反馈增益

Firstly, we must enter the feedforward and feedback gains which we obtained from the identification tool.

首先输入从 SysId 工具获得的前馈和反馈增益。

Note / 说明

Feedforward and feedback gains do not, in general, transfer across robots. Do not use the gains from this tutorial for your own robot.

通常，不同机器人之间的前馈和反馈增益不能直接通用。不要把本教程中的增益用于自己的机器人。

```
// These are example values only - DO NOT USE THESE FOR YOUR OWN ROBOT!
// These characterization values MUST be determined either experimentally or theoretically
// for *your* robot's drive.
// The Robot Characterization Toolsuite provides a convenient tool for obtaining these
// values for your robot.
public static final double ksVolts = 0.22;
public static final double kvVoltSecondsPerMeter = 1.98;
public static final double kaVoltSecondsSquaredPerMeter = 0.2;

// Example value only - as above, this must be tuned for your drive!
public static final double kPDriveVel = 8.5;
```

DifferentialDriveKinematics / DifferentialDriveKinematics

Additionally, we must create an instance of the DifferentialDriveKinematics class, which allows us to use the trackwidth (i.e. horizontal distance between the wheels) of the robot to convert from chassis speeds to wheel speeds. As elsewhere, we keep our units in meters.

此外，需要创建 DifferentialDriveKinematics 类的实例。它可以利用机器人轮距（即两侧车轮之间的水平距离）将底盘速度转换为车轮速度。与其他地方一样，单位保持为米。

```
public static final double kTrackwidthMeters = 0.69;
public static final DifferentialDriveKinematics kDriveKinematics =
    new DifferentialDriveKinematics(kTrackwidthMeters);
```

Max Trajectory Velocity/Acceleration / 轨迹最大速度 / 加速度

We must also decide on a nominal max acceleration and max velocity for the robot during path-following. The maximum velocity value should be set somewhat below the nominal free-speed of the robot. Due to the later use of the `DifferentialDriveVoltageConstraint`, the maximum acceleration value is not extremely crucial.

我们还需要决定机器人在路径跟随期间的标称最大加速度和最大速度。最大速度应略低于机器人的标称空载速度。由于后续会使用 `DifferentialDriveVoltageConstraint`，最大加速度的数值并非极其关键。

Warning / 警告

Max velocity and acceleration, as defined here, are applied only during trajectory generation. They do not limit the `RamseteCommand` itself, which may give values to the `DriveSubsystem` that can cause the robot to greatly exceed these velocities and accelerations.

这里定义的最大速度和加速度仅在生成轨迹时应用。它们不会限制 `RamseteCommand` 本身；后者可能向 `DriveSubsystem` 提供使机器人远超这些速度和加速度的值。

```
public static final double kMaxSpeedMetersPerSecond = 3;
public static final double kMaxAccelerationMetersPerSecondSquared = 1;
```

Ramsete Parameters / Ramsete 参数

Finally, we must include a pair of parameters for the `RAMSETE` controller. The values `b` and `zeta` shown below should work well for most robots, provided distances have been correctly measured in meters.

最后，还需为 `RAMSETE` 控制器加入一对参数。只要距离以米正确测量，下方所示的 `b` 和 `zeta` 值应适用于大多数机器人。

Larger values of `b` make convergence more aggressive like a proportional term whereas larger values of `zeta` provide more damping in the response. These controller gains only dictate how the controller will output adjusted velocities. It does NOT affect the actual velocity tracking of the robot. This means that these controller gains are generally robot-agnostic.

较大的 `b` 值会像比例项一样使收敛更激进，较大的 `zeta` 值则会为响应提供更多阻尼。这些控制器增益只决定控制器如何输出修正后的速度，不会影响机器人实际的速度跟踪。这意味着这些控制器增益通常与具体机器人无关。

Note / 说明

Gains of 2.0 and 0.7 for `b` and `zeta` have been tested repeatedly to produce desirable results when all units were in meters. As such, a zero-argument constructor for `RamseteController` exists with gains defaulted to these values.

在所有单位均为米的情况下，`b` 和 `zeta` 分别取 2.0 和 0.7 已经反复测试，并得到了理想结果。因此，`RamseteController` 提供了无参数构造函数，默认使用这些增益。

```
// Reasonable baseline values for a RAMSETE follower in units of meters and seconds
public static final double kRamseteB = 2;
public static final double kRamseteZeta = 0.7;
```

Step 3: Creating a Drive Subsystem / 步骤 3: 创建底盘 subsystem

Now that our drive is characterized, it is time to start writing our robot code proper. As mentioned before, we will use the [command-based](#) framework for our robot code. Accordingly, our first step is to write a suitable drive [subsystem](#) class.

现在底盘已完成表征，可以开始编写实际的机器人代码。如前所述，我们会使用 [Command-Based](#) 框架。因此，第一步是编写合适的底盘 [subsystem](#) 类。

The full drive class from the RamseteCommand Example Project ([Java](#)) can be seen below. The rest of the article will describe the steps involved in writing this class.

下方展示 RamseteCommand 示例项目 ([Java](#)) 中的完整底盘类。本文其余部分将介绍编写该类的各个步骤。

```
package edu.wpi.first.wpilibj.examples.ramsetecommand.subsystems;

import edu.wpi.first.math.geometry.Pose2d;
import edu.wpi.first.math.kinematics.DifferentialDriveOdometry;
import edu.wpi.first.math.kinematics.DifferentialDriveWheelSpeeds;
import edu.wpi.first.util.sendable.SendableRegistry;
import edu.wpi.first.wpilibj.ADXRS450_Gyro;
import edu.wpi.first.wpilibj.Encoder;
import edu.wpi.first.wpilibj.drive.DifferentialDrive;
import edu.wpi.first.wpilibj.examples.ramsetecommand.Constants.DriveConstants;
import edu.wpi.first.wpilibj.motorcontrol.PWMSparkMax;
import edu.wpi.first.wpilibj2.command.SubsystemBase;

public class DriveSubsystem extends SubsystemBase {
    // The motors on the left side of the drive.
    private final PWMSparkMax m_leftLeader = new PWMSparkMax(DriveConstants.kLeftMotor1Port);
    private final PWMSparkMax m_leftFollower = new PWMSparkMax(DriveConstants.kLeftMotor2Port);

    // The motors on the right side of the drive.
    private final PWMSparkMax m_rightLeader = new PWMSparkMax(DriveConstants.kRightMotor1Port);
    private final PWMSparkMax m_rightFollower = new PWMSparkMax(DriveConstants.kRightMotor2Port);

    // The robot's drive
    private final DifferentialDrive m_drive =
        new DifferentialDrive(m_leftLeader::set, m_rightLeader::set);

    // The left-side drive encoder
    private final Encoder m_leftEncoder =
        new Encoder(
            DriveConstants.kLeftEncoderPorts[0],
            DriveConstants.kLeftEncoderPorts[1],
            DriveConstants.kLeftEncoderReversed);

    // The right-side drive encoder
    private final Encoder m_rightEncoder =
        new Encoder(
            DriveConstants.kRightEncoderPorts[0],
            DriveConstants.kRightEncoderPorts[1],
            DriveConstants.kRightEncoderReversed);

    // The gyro sensor
    private final ADXRS450_Gyro m_gyro = new ADXRS450_Gyro();

    // Odometry class for tracking robot pose
    private final DifferentialDriveOdometry m_odometry;

    /** Creates a new DriveSubsystem. */
    public DriveSubsystem() {
```

```

SendableRegistry.addChild(m_drive, m_leftLeader);
SendableRegistry.addChild(m_drive, m_rightLeader);

m_leftLeader.addFollower(m_leftFollower);
m_rightLeader.addFollower(m_rightFollower);

// We need to invert one side of the drivetrain so that positive voltages
// result in both sides moving forward. Depending on how your robot's
// gearbox is constructed, you might have to invert the left side instead.
m_rightLeader.setInverted(true);

// Sets the distance per pulse for the encoders
m_leftEncoder.setDistancePerPulse(DriveConstants.kEncoderDistancePerPulse);
m_rightEncoder.setDistancePerPulse(DriveConstants.kEncoderDistancePerPulse);

resetEncoders();
m_odometry =
    new DifferentialDriveOdometry(
        m_gyro.getRotation2d(), m_leftEncoder.getDistance(), m_rightEncoder.getDistance());
}

@Override
public void periodic() {
    // Update the odometry in the periodic block
    m_odometry.update(
        m_gyro.getRotation2d(), m_leftEncoder.getDistance(), m_rightEncoder.getDistance());
}

/**
 * Returns the currently-estimated pose of the robot.
 *
 * @return The pose.
 */
public Pose2d getPose() {
    return m_odometry.getPoseMeters();
}

/**
 * Returns the current wheel speeds of the robot.
 *
 * @return The current wheel speeds.
 */
public DifferentialDriveWheelSpeeds getWheelSpeeds() {
    return new DifferentialDriveWheelSpeeds(m_leftEncoder.getRate(), m_rightEncoder.getRate());
}

/**
 * Resets the odometry to the specified pose.
 *
 * @param pose The pose to which to set the odometry.
 */
public void resetOdometry(Pose2d pose) {
    m_odometry.resetPosition(
        m_gyro.getRotation2d(), m_leftEncoder.getDistance(), m_rightEncoder.getDistance(), pose);
}

/**
 * Drives the robot using arcade controls.
 *
 * @param fwd the commanded forward movement
 * @param rot the commanded rotation
 */
public void arcadeDrive(double fwd, double rot) {
    m_drive.arcadeDrive(fwd, rot);
}

/**
 * Controls the left and right sides of the drive directly with voltages.
 *

```

```

    * @param leftVolts the commanded left output
    * @param rightVolts the commanded right output
    */
    public void tankDriveVolts(double leftVolts, double rightVolts) {
        m_leftLeader.setVoltage(leftVolts);
        m_rightLeader.setVoltage(rightVolts);
        m_drive.feed();
    }

    /** Resets the drive encoders to currently read a position of 0. */
    public void resetEncoders() {
        m_leftEncoder.reset();
        m_rightEncoder.reset();
    }

    /**
     * Gets the average distance of the two encoders.
     *
     * @return the average of the two encoder readings
     */
    public double getAverageEncoderDistance() {
        return (m_leftEncoder.getDistance() + m_rightEncoder.getDistance()) / 2.0;
    }

    /**
     * Gets the left drive encoder.
     *
     * @return the left drive encoder
     */
    public Encoder getLeftEncoder() {
        return m_leftEncoder;
    }

    /**
     * Gets the right drive encoder.
     *
     * @return the right drive encoder
     */
    public Encoder getRightEncoder() {
        return m_rightEncoder;
    }

    /**
     * Sets the max output of the drive. Useful for scaling the drive to drive more slowly.
     *
     * @param maxOutput the maximum output to which the drive will be constrained
     */
    public void setMaxOutput(double maxOutput) {
        m_drive.setMaxOutput(maxOutput);
    }

    /** Zeroes the heading of the robot. */
    public void zeroHeading() {
        m_gyro.reset();
    }

    /**
     * Returns the heading of the robot.
     *
     * @return the robot's heading in degrees, from -180 to 180
     */
    public double getHeading() {
        return m_gyro.getRotation2d().getDegrees();
    }

    /**
     * Returns the turn rate of the robot.
     *
     * @return The turn rate of the robot, in degrees per second

```

```

    */
    public double getTurnRate() {
        return -m_gyro.getRate();
    }
}

```

Configuring the Drive Encoders / 配置底盘编码器

The drive encoders measure the rotation of the wheels on each side of the drive. To properly configure the encoders, we need to specify two things: the ports the encoders are plugged into, and the distance per encoder pulse. Then, we need to write methods allowing access to the encoder values from code that uses the subsystem.

底盘编码器测量底盘两侧车轮的旋转。要正确配置编码器，需要指定两项内容：编码器连接的端口，以及每个编码器脉冲对应的距离。然后，还需编写方法，让使用该 subsystem 的代码能够访问编码器数值。

Encoder Ports / 编码器端口

The encoder ports are specified in the encoder's constructor, like so:

编码器端口在其构造函数中指定，如下所示：

```

// The left-side drive encoder
private final Encoder m_leftEncoder =
    new Encoder(
        DriveConstants.kLeftEncoderPorts[0],
        DriveConstants.kLeftEncoderPorts[1],
        DriveConstants.kLeftEncoderReversed);

// The right-side drive encoder
private final Encoder m_rightEncoder =
    new Encoder(
        DriveConstants.kRightEncoderPorts[0],
        DriveConstants.kRightEncoderPorts[1],
        DriveConstants.kRightEncoderReversed);

```

Encoder Distance per Pulse / 每个编码器脉冲对应的距离

The distance per pulse is specified by calling the encoder's `setDistancePerPulse` method. Note that for the WPILib Encoder class, “pulse” refers to a full encoder cycle (i.e. four edges), and thus will be 1/4 the value that was specified in the SysId config. Remember, as well, that the distance should be measured in meters!

通过调用编码器的 `setDistancePerPulse` 方法，指定每个脉冲对应的距离。注意，对 WPILib Encoder 类而言，“脉冲”指一个完整的编码器周期（即四个边沿），因此这里的数值应为 SysId 配置中所指定值的 1/4。还要记住，距离应以米计量！

```

m_leftEncoder.setDistancePerPulse(DriveConstants.kEncoderDistancePerPulse);
m_rightEncoder.setDistancePerPulse(DriveConstants.kEncoderDistancePerPulse);

```

Encoder Accessor Method / 编码器访问方法

To access the values measured by the encoders, we include the following method:

为了访问编码器测得的数值，加入以下方法：

Important / 重要

The returned velocities must be in meters! Because we configured the distance per pulse on the encoders above, calling `getRate()` will automatically apply the conversion factor from encoder units to meters. If you are not using WPILib's Encoder class, you must perform this conversion either through the respective vendor's API or by manually multiplying by a conversion factor.

返回的速度必须采用米！由于上文已经为编码器配置了每个脉冲对应的距离，调用 `getRate()` 会自动应用从编码器单位到米的换算系数。如果没有使用 WPILib 的 Encoder 类，则必须通过相应厂商的 API，或手动乘以换算系数来完成该转换。

译注：原文将返回的速度单位写作“米”。按 WPILib Encoder 的 `getRate()` 定义，若 `setDistancePerPulse()` 采用米，返回值的单位应为 m/s。

```
/**
 * Returns the current wheel speeds of the robot.
 *
 * @return The current wheel speeds.
 */
public DifferentialDriveWheelSpeeds getWheelSpeeds() {
    return new DifferentialDriveWheelSpeeds(m_leftEncoder.getRate(), m_rightEncoder.getRate());
}
```

We wrap the measured encoder values in a `DifferentialDriveWheelSpeeds` object for easier integration with the `RamseteCommand` class later on.

我们把测得的编码器数值封装在 `DifferentialDriveWheelSpeeds` 对象中，以便之后与 `RamseteCommand` 类集成。

Configuring the Gyroscope / 配置陀螺仪

The gyroscope measures the rate of change of the robot's heading (which can then be integrated to provide a measurement of the robot's heading relative to when it first turned on). In our example, we use the [Analog Devices ADXRS450 FRC Gyro Board](#), which was included in the kit of parts for several years:

陀螺仪测量机器人朝向的变化率（对其积分后，可得到机器人相对于开机时朝向的测量值）。在示例中，我们使用 [Analog Devices ADXRS450 FRC Gyro Board](#)，它曾连续多年被包含在零件套件中：

```
// The gyro sensor
private final ADXRS450_Gyro m_gyro = new ADXRS450_Gyro();
```

Gyroscope Accessor Method / 陀螺仪访问方法

To access the current heading measured by the gyroscope, we include the following method:

为了访问陀螺仪测得的当前朝向，加入以下方法：

```
/**
 * Returns the heading of the robot.
 *
 * @return the robot's heading in degrees, from -180 to 180
 */
public double getHeading() {
    return m_gyro.getRotation2d().getDegrees();
}
```

Configuring the Odometry / 配置 odometry

Now that we have our encoders and gyroscope configured, it is time to set up our drive subsystem to automatically compute its position from the encoder and gyroscope readings.

编码器和陀螺仪配置完成后，需要设置底盘 subsystem，使其根据编码器和陀螺仪读数自动计算位置。

First, we create a member instance of the `DifferentialDriveOdometry` class:

首先创建 `DifferentialDriveOdometry` 类的成员实例：

```
// Odometry class for tracking robot pose
private final DifferentialDriveOdometry m_odometry;
```

Then we initialize the `DifferentialDriveOdometry`.

然后初始化 `DifferentialDriveOdometry`。

```
m_odometry =
    new DifferentialDriveOdometry(
        m_gyro.getRotation2d(), m_leftEncoder.getDistance(), m_rightEncoder.getDistance());
```

Updating the Odometry / 更新 odometry

The odometry class must be regularly updated to incorporate new readings from the encoder and gyroscope. We accomplish this inside the subsystem's periodic method, which is automatically called once per main loop iteration:

必须定期更新 odometry 类，使其纳入编码器和陀螺仪的新读数。我们在 subsystem 的 periodic 方法中完成此操作，该方法在主循环的每次迭代中自动调用一次：

```
@Override
public void periodic() {
    // Update the odometry in the periodic block
    m_odometry.update(
        m_gyro.getRotation2d(), m_leftEncoder.getDistance(), m_rightEncoder.getDistance());
}
```

Odometry Accessor Method / odometry 访问方法

To access the robot's current computed pose, we include the following method:

为了访问机器人当前计算出的 pose，加入以下方法：

```
/**
 * Returns the currently-estimated pose of the robot.
 *
 * @return The pose.
 */
public Pose2d getPose() {
    return m_odometry.getPoseMeters();
}
```

Important / 重要

Before running a `RamseteCommand`, teams are strongly encouraged to deploy and test the odometry code alone, with values sent to the SmartDashboard or Shuffleboard during the `DriveSubsystem`'s `periodic()`. This odometry must be correct for a `RamseteCommand` to successfully work, as sign or unit errors can cause a robot to move at high speeds in unpredictable directions.

在运行 `RamseteCommand` 之前，强烈建议队伍单独部署并测试 odometry 代码，并在 `DriveSubsystem` 的 `periodic()` 中将数值发送到 SmartDashboard 或 Shuffleboard。odometry 必须正确，`RamseteCommand` 才能正常工作；符号方向或单位错误可能使机器人以高速度朝不可预测的方向运动。

Voltage-Based Drive Method / 基于电压的底盘驱动方法

Finally, we must include one additional method – a method that allows us to set the voltage to each side of the drive using the `setVoltage()` method of the `MotorController` interface. The default WPILib drive class does not include this functionality, so we must write it ourselves:

最后，还必须加入一个方法：使用 `MotorController` 接口的 `setVoltage()` 方法，设置底盘两侧的电压。WPILib 默认底盘类不包含此功能，因此需要自行编写：

```
/**
 * Controls the left and right sides of the drive directly with voltages.
 *
 * @param leftVolts the commanded left output
 * @param rightVolts the commanded right output
 */
public void tankDriveVolts(double leftVolts, double rightVolts) {
    m_leftLeader.setVoltage(leftVolts);
    m_rightLeader.setVoltage(rightVolts);
    m_drive.feed();
}
```

It is very important to use the `setVoltage()` method rather than the ordinary `set()` method, as this will automatically compensate for battery “voltage sag” during operation. Since our feedforward voltages are physically-meaningful (as they are based on measured identification data), this is essential to ensuring their accuracy.

使用 `setVoltage()` 方法而非普通 `set()` 方法非常重要，因为它会在运行期间自动补偿电池“电压下垂”。由于前馈电压具有物理意义（它们基于测得的 `SysId` 数据），这种补偿对于保证准确性至关重要。

Warning / 警告

`RamseteCommand` itself does not internally enforce any speed or acceleration limits before providing motor voltage parameters to this method. During initial code development, teams are strongly encouraged to apply both maximum and minimum bounds on the input variables before passing these values to `setVoltage()` while ensuring the trajectory velocity and acceleration are achievable. For example, generate a trajectory with a little less than half of the Robot's maximum velocity and limit voltage to 6 volts.

`RamseteCommand` 本身在将电机电压参数提供给此方法前，不会在内部强制实施任何速度或加速度限制。在初始代码开发阶段，强烈建议队伍在把输入变量传给 `setVoltage()` 前，对其设置上限和下限，同时确保轨迹速度与加速度可实现。例如，生成最大速度略低于机器人最高速度一半的轨迹，并将电压限制为 6 伏特。

Step 4: Creating and Following a Trajectory / 步骤 4：创建并跟随轨迹

With our drive subsystem written, it is now time to generate a trajectory and write an autonomous command to follow it.

底盘 subsystem 编写完成后，就可以生成轨迹，并编写用于跟随它的自主 Command。

As per the [standard command-based project structure](#), we will do this in the `getAutonomousCommand` method of the `RobotContainer` class. The full method from the [RamseteCommand Example Project \(Java\)](#) can be seen below. The rest of the article will break down the different parts of the method in more detail.

按照[标准 Command-Based 项目结构](#)，我们将在 `RobotContainer` 类的 `getAutonomousCommand` 方法中完成此操作。下方展示 `RamseteCommand` 示例项目 (Java) 中的完整方法。本文其余部分将更详细地拆解该方法的各个部分。

```
/**
 * Use this to pass the autonomous command to the main {@link Robot} class.
 *
 * @return the command to run in autonomous
 */
public Command getAutonomousCommand() {
    // Create a voltage constraint to ensure we don't accelerate too fast
    var autoVoltageConstraint =
        new DifferentialDriveVoltageConstraint(
            new SimpleMotorFeedforward(
                DriveConstants.ksVolts,
                DriveConstants.kvVoltSecondsPerMeter,
                DriveConstants.kaVoltSecondsSquaredPerMeter),
            DriveConstants.kDriveKinematics,
            10);

    // Create config for trajectory
    TrajectoryConfig config =
        new TrajectoryConfig(
            AutoConstants.kMaxSpeedMetersPerSecond,
            AutoConstants.kMaxAccelerationMetersPerSecondSquared)
        // Add kinematics to ensure max speed is actually obeyed
        .setKinematics(DriveConstants.kDriveKinematics)
        // Apply the voltage constraint
        .addConstraint(autoVoltageConstraint);

    // An example trajectory to follow. All units in meters.
    Trajectory exampleTrajectory =
        TrajectoryGenerator.generateTrajectory(
            // Start at the origin facing the +X direction
            new Pose2d(0, 0, new Rotation2d(0)),
            // Pass through these two interior waypoints, making an 's' curve path
            List.of(new Translation2d(1, 1), new Translation2d(2, -1)),
            // End 3 meters straight ahead of where we started, facing forward
            new Pose2d(3, 0, new Rotation2d(0)),
            // Pass config
            config);

    RamseteCommand ramseteCommand =
        new RamseteCommand(
            exampleTrajectory,
            m_robotDrive::getPose,
            new RamseteController(AutoConstants.kRamseteB, AutoConstants.kRamseteZeta),
            new SimpleMotorFeedforward(
                DriveConstants.ksVolts,
                DriveConstants.kvVoltSecondsPerMeter,
                DriveConstants.kaVoltSecondsSquaredPerMeter),
            DriveConstants.kDriveKinematics,
```

```

        m_robotDrive::getWheelSpeeds,
        new PIDController(DriveConstants.kPDriveVel, 0, 0),
        new PIDController(DriveConstants.kPDriveVel, 0, 0),
        // RamseteCommand passes volts to the callback
        m_robotDrive::tankDriveVolts,
        m_robotDrive);

    // Reset odometry to the initial pose of the trajectory, run path following
    // command, then stop at the end.
    return Commands.runOnce(() -> m_robotDrive.resetOdometry(exampleTrajectory.getInitialPose()))
        .andThen(ramseteCommand)
        .andThen(Commands.runOnce(() -> m_robotDrive.tankDriveVolts(0, 0)));
}
}

```

Configuring the Trajectory Constraints / 配置轨迹约束

First, we must set some configuration parameters for the trajectory which will ensure that the generated trajectory is followable.

首先，需要为轨迹设置一些配置参数，以确保生成的轨迹能够被机器人跟随。

Creating a Voltage Constraint / 创建电压约束

The first piece of configuration we will need is a voltage constraint. This will ensure that the generated trajectory never commands the robot to go faster than it is capable of achieving with the given voltage supply:

首先需要的配置项是电压约束。它可确保生成的轨迹不会要求机器人在给定供电电压下以超出自身能力的速度行驶：

```

// Create a voltage constraint to ensure we don't accelerate too fast
var autoVoltageConstraint =
    new DifferentialDriveVoltageConstraint(
        new SimpleMotorFeedforward(
            DriveConstants.ksVolts,
            DriveConstants.kvVoltSecondsPerMeter,
            DriveConstants.kaVoltSecondsSquaredPerMeter),
        DriveConstants.kDriveKinematics,
        10);

```

Notice that we set the maximum voltage to 10V, rather than the nominal battery voltage of 12V. This gives us some “headroom” to deal with “voltage sag” during operation.

请注意，我们把最大电压设为 10V，而不是电池标称电压 12V。这样可留出一些余量，应对运行期间的“电压下垂”。

Creating the Configuration / 创建配置

Now that we have our voltage constraint, we can create our `TrajectoryConfig` instance, which wraps together all of our path constraints:

有了电压约束后，就可以创建 `TrajectoryConfig` 实例，把所有路径约束组合起来：

```

// Create config for trajectory
TrajectoryConfig config =
    new TrajectoryConfig(
        AutoConstants.kMaxSpeedMetersPerSecond,
        AutoConstants.kMaxAccelerationMetersPerSecondSquared)
    // Add kinematics to ensure max speed is actually obeyed
    .setKinematics(DriveConstants.kDriveKinematics)
    // Apply the voltage constraint

```

```
.addConstraint(autoVoltageConstraint);
```

Generating the Trajectory / 生成轨迹

With our trajectory configuration in hand, we are now ready to generate our trajectory. For this example, we will be generating a “clamped cubic” trajectory – this means we will specify full robot poses at the endpoints, and positions only for interior waypoints (also known as “knot points”). As elsewhere, all distances are in meters.

轨迹配置准备好后，就可以生成轨迹。在本例中，我们将生成一条“端点切向量已约束的三次样条”轨迹：这意味着端点处指定完整的机器人 pose，而内部路径点（也称为“结点”）只指定位置。与其他地方一样，所有距离均以米为单位。

```
// An example trajectory to follow. All units in meters.
Trajectory exampleTrajectory =
    TrajectoryGenerator.generateTrajectory(
        // Start at the origin facing the +X direction
        new Pose2d(0, 0, new Rotation2d(0)),
        // Pass through these two interior waypoints, making an 's' curve path
        List.of(new Translation2d(1, 1), new Translation2d(2, -1)),
        // End 3 meters straight ahead of where we started, facing forward
        new Pose2d(3, 0, new Rotation2d(0)),
        // Pass config
        config);
```

Note / 说明

Instead of generating the trajectory on the roboRIO as outlined above, one can also [import a PathWeaver JSON](#).

除了按上文方式在 roboRIO 上生成轨迹，也可以[导入 PathWeaver JSON](#)。

Creating the RamseteCommand / 创建 RamseteCommand

We will first reset our robot’s pose to the starting pose of the trajectory. This ensures that the robot’s location on the coordinate system and the trajectory’s starting position are the same.

首先将机器人的 pose 重置为轨迹起始 pose。这能确保机器人在坐标系中的位置与轨迹起点一致。

```
// Reset odometry to the initial pose of the trajectory, run path following
// command, then stop at the end.
return Commands.runOnce(() -> m_robotDrive.resetOdometry(exampleTrajectory.getInitialPose()))
```

It is very important that the initial robot pose match the first pose in the trajectory. For the purposes of our example, the robot will be reliably starting at a position of (0,0) with a heading of 0. In actual use, however, it is probably not desirable to base your coordinate system on the robot position, and so the starting position for both the robot and the trajectory should be set to some other value. If you wish to use a trajectory that has been defined in robot-centric coordinates in such a situation, you can transform it to be relative to the robot’s current pose using the `transformBy` method ([Java](#)). For more information about transforming trajectories, see [Transforming Trajectories](#).

机器人初始 pose 与轨迹的第一个 pose 一致非常重要。在本例中，机器人会可靠地从位置 (0,0)、朝向 0 起步。但在实际应用中，通常不宜以机器人位置作为坐标系基准，因此机器人和轨迹的起点都应设为其他数值。如果希望在这种情况下使用以机器人为中心的坐标定义的轨迹，可使用 `transformBy` 方法 ([Java](#)) 将其转换为相对于机器人当前 pose 的轨迹。有关轨迹变换的更多信息，请参阅[变换轨迹](#)。

Now that we have a trajectory, we can create a command that, when executed, will follow that trajectory. To do this, we use the `RamseteCommand` class (Java)

有了轨迹后，就可以创建一个在执行时跟随该轨迹的 Command。为此，我们使用 `RamseteCommand` 类 (Java)。

```
RamseteCommand ramseteCommand =
    new RamseteCommand(
        exampleTrajectory,
        m_robotDrive::getPose,
        new RamseteController(AutoConstants.kRamseteB, AutoConstants.kRamseteZeta),
        new SimpleMotorFeedforward(
            DriveConstants.kSVolts,
            DriveConstants.kVVoltsSecondsPerMeter,
            DriveConstants.kAVoltsSecondsSquaredPerMeter),
        DriveConstants.kDriveKinematics,
        m_robotDrive::getWheelSpeeds,
        new PIDController(DriveConstants.kPDriveVel, 0, 0),
        new PIDController(DriveConstants.kPDriveVel, 0, 0),
        // RamseteCommand passes volts to the callback
        m_robotDrive::tankDriveVolts,
        m_robotDrive);
```

This declaration is fairly substantial, so we'll go through it argument-by-argument:

这一声明包含很多内容，下面按参数逐一说明：

1. The trajectory: This is the trajectory to be followed; accordingly, we pass the command the trajectory we just constructed in our earlier steps.
1. 轨迹：要跟随的轨迹。因此，将先前步骤中刚构造的轨迹传给 Command。
2. The pose supplier: This is a method reference (or lambda) to the `drive subsystem method that returns the pose`. The RAMSETE controller needs the current pose measurement to determine the required wheel outputs.
2. pose 提供者：一个指向返回 pose 的底盘 subsystem 方法的方法引用（或 lambda）。RAMSETE 控制器需要当前 pose 测量值来确定所需的车轮输出。
3. The RAMSETE controller: This is the `RamseteController` object (Java) that will perform the path-following computation that translates the current measured pose and trajectory state into a chassis speed setpoint.
3. RAMSETE 控制器：RamseteController 对象 (Java)。它执行路径跟随计算，将当前测得的 pose 和轨迹状态转换为底盘速度 setpoint。
4. The drive feedforward: This is a `SimpleMotorFeedforward` object (Java) that will automatically perform the correct feedforward calculation with the feedforward gains (kS, kV, and kA) that we obtained from the drive identification tool.
4. 底盘前馈：SimpleMotorFeedforward 对象 (Java)。它使用从底盘 SysId 工具获得的前馈增益 (kS、kV 和 kA)，自动完成正确的前馈计算。
5. The drive kinematics: This is the `DifferentialDriveKinematics` object (Java) that we constructed earlier in our constants file, and will be used to convert chassis speeds to wheel speeds.
5. 底盘运动学：DifferentialDriveKinematics 对象 (Java)。它是在前面的常数文件中构造的，用于将底盘速度转换为车轮速度。
6. The wheel speed supplier: This is a method reference (or lambda) to the `drive subsystem method that returns the wheel speeds`
6. 车轮速度提供者：一个指向返回车轮速度的底盘 subsystem 方法的方法引用（或 lambda）。

7. The left-side PIDController: This is the PIDController object (Java) that will track the left-side wheel speed setpoint, using the P gain that we obtained from the drive identification tool.
7. 左侧 PIDController: PIDController 对象 (Java)。它使用从底盘 SysId 工具获得的 P 增益跟踪左侧车轮速度 setpoint。
8. The right-side PIDController: This is the PIDController object (Java) that will track the right-side wheel speed setpoint, using the P gain that we obtained from the drive identification tool.
8. 右侧 PIDController: PIDController 对象 (Java)。它使用从底盘 SysId 工具获得的 P 增益跟踪右侧车轮速度 setpoint。
9. The output consumer: This is a method reference (or lambda) to the `drive subsystem method that passes the voltage outputs to the drive motors`.
9. 输出消费者: 一个指向将电压输出传给底盘电机的底盘 `subsystem 方法` 的方法引用 (或 lambda)。
10. The robot drive: This is the drive subsystem itself, included to ensure the command does not operate on the drive at the same time as any other command that uses the drive.
10. 机器人底盘: 底盘 `subsystem` 本身。将其加入 Command, 以确保该 Command 不会与其他使用底盘的 Command 同时控制底盘。

Finally, note that we append a final “stop” command in sequence after the path-following command, to ensure that the robot stops moving at the end of the trajectory.

最后请注意, 我们在路径跟随 Command 之后顺序附加了一个“停止”Command, 以确保机器人在轨迹结束时停止运动。

Video / 视频

If all has gone well, your robot's autonomous routine should look something like this:

如果一切顺利, 机器人的自主例程应与下面的演示类似:

Video / 视频: <https://www.youtube.com/watch?v=yVmJD0E3M2Y>

Video / 视频: <https://www.youtube.com/watch?v=FLn1bFqIkL0>

PathWeaver / PathWeaver

Warning / 警告

PathWeaver is deprecated and will be removed for 2027. Users may find [Choreo](#) or [PathPlanner](#) more useful. They both have an intuitive user interface and swerve support.

PathWeaver 已被弃用，并将于 2027 年移除。用户可能会发现 [Choreo](#) 或 [PathPlanner](#) 更有用。两者都具备直观的用户界面并支持 swerve。

- [Introduction to PathWeaver](#)
- [PathWeaver 简介](#)
- [Creating a Pathweaver Project](#)
- [创建 PathWeaver 项目](#)
- [Visualizing PathWeaver Trajectories](#)
- [可视化 PathWeaver 轨迹](#)
- [Creating Autonomous Routines](#)
- [创建自主例程](#)
- [Importing a PathWeaver JSON](#)
- [导入 PathWeaver JSON](#)
- [Adding field images to PathWeaver](#)
- [向 PathWeaver 添加场地图像](#)

Introduction to PathWeaver / PathWeaver 简介

Warning / 警告

PathWeaver is deprecated and will be removed for 2027. Users may find [Choreo](#) or [PathPlanner](#) more useful. They both have an intuitive user interface and swerve support.

PathWeaver 已被弃用，并将于 2027 年移除。用户可能会发现 [Choreo](#) 或 [PathPlanner](#) 更有用。两者都具备直观的用户界面并支持 swerve。

Autonomous is an important section of the match; it is exciting when robots do impressive things in autonomous. In order to score, the robot usually need to go somewhere. The faster the robot arrives at that location, the sooner it can score points! The traditional method for autonomous is driving in a straight line, turning to a certain angle, and driving in a straight line again. This approach works fine, but the robot spends a non-negligible amount of time stopping and starting again after each straight line and turn.

自主阶段是比赛的重要环节；机器人在自主阶段完成出色动作令人兴奋。为了得分，机器人通常需要移动到某个位置。到达得越快，就能越早得分！传统的自主移动方式是先沿直线行驶、转到指定角度，再沿直线行驶。这种方式可行，但机器人在每段直行和转向之后停止并重新启动，会耗费不可忽略的时间。

A more advanced approach to autonomous is called “path planning”. Instead of driving in a straight line and turning once the line is complete, the robot continuously moves, driving with a curve-like motion. This can reduce turning stoppage time.

一种更高级的自主移动方式称为“路径规划”。机器人不再走完一段直线后才转向，而是持续运动，沿曲线行驶。这可以减少为转向而停止的时间。

WPILib contains a trajectory generation suite that can be used by teams to generate and follow trajectories. This series of articles will go over how to generate and visualize trajectories using PathWeaver. For a comprehensive tutorial on following trajectories, please visit the [end-to-end trajectory tutorial](#).

WPILib 提供轨迹生成工具集，队伍可以用它生成并跟随轨迹。本系列文章将介绍如何使用 PathWeaver 生成轨迹并将其可视化。有关跟随轨迹的完整教程，请参阅[端到端轨迹教程](#)。

Note / 说明

[Trajectory following](#) code is required to use PathWeaver. We recommend that you start with Trajectory following and get that working with simple paths. From there you can continue on to testing more complicated paths generated by PathWeaver.

使用 PathWeaver 需要[轨迹跟随](#)代码。建议先从轨迹跟随入手，让它在简单路径上正常工作，然后再测试 PathWeaver 生成的更复杂路径。

Creating a Pathweaver Project / 创建 PathWeaver 项目

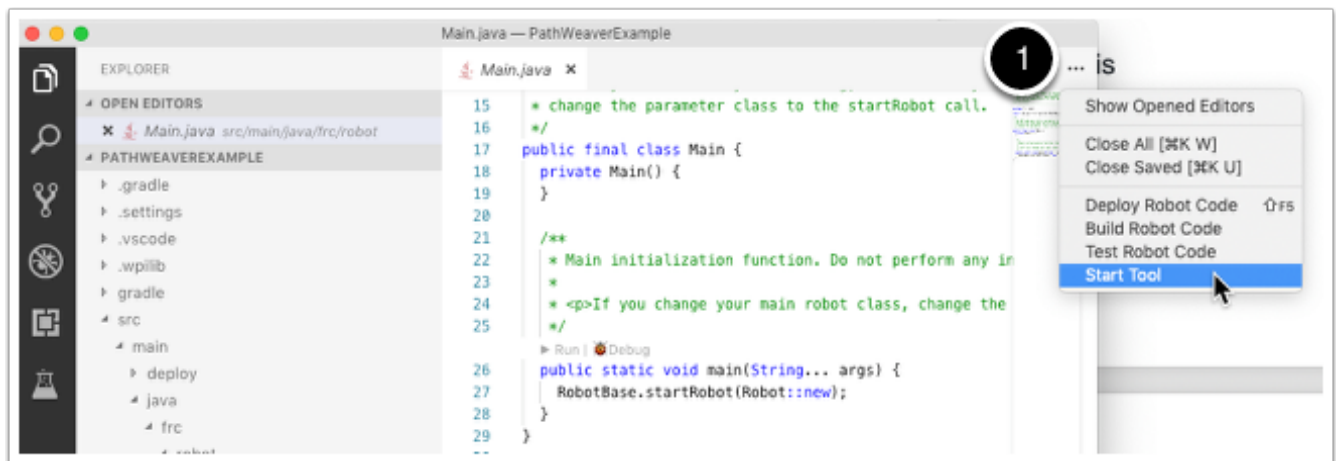
PathWeaver is the tool used to draw the paths for a robot to follow. The paths for a single program are stored in a PathWeaver project.

PathWeaver 是绘制供机器人跟随的路径的工具。单个程序的路径存储在一个 PathWeaver 项目中。

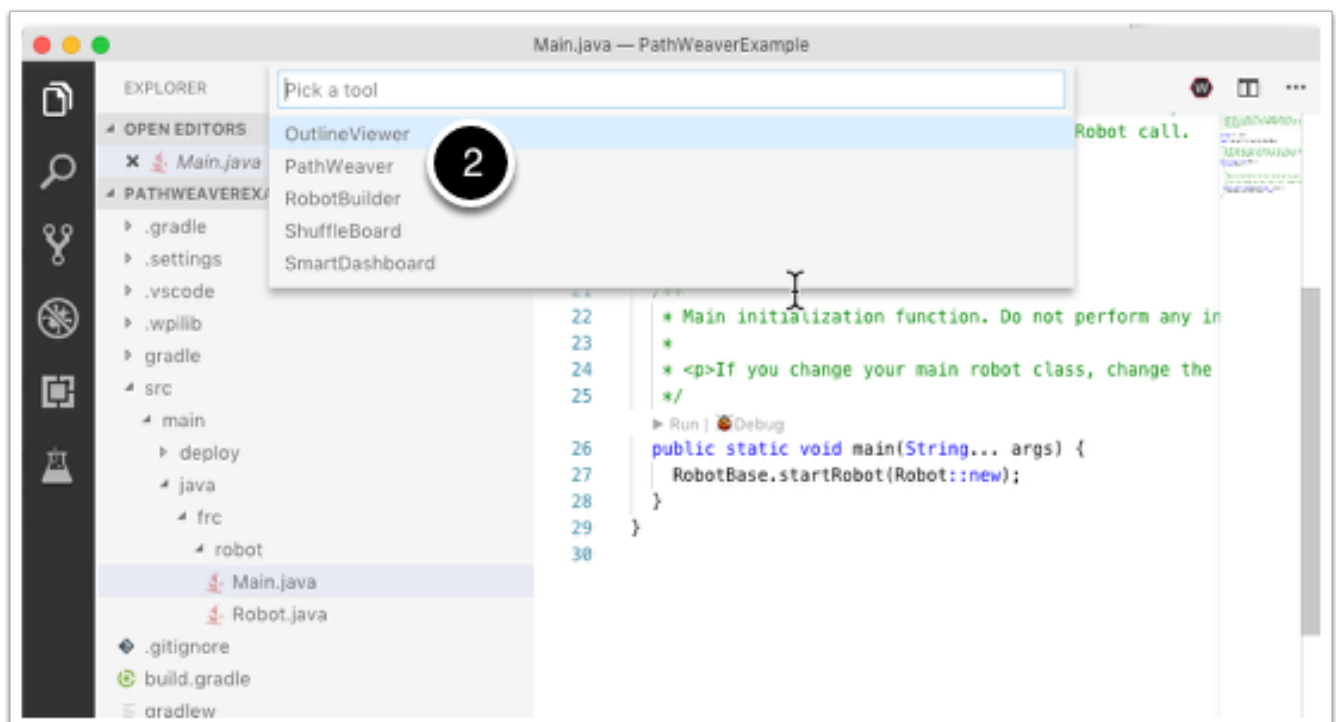
Starting PathWeaver / 启动 PathWeaver

PathWeaver is started by clicking on the ellipsis icon in the top right of the corner of the Visual Studio Code interface. You must select a source file from the WPILib project to see the icon. Then click on “Start tool” and then click on “PathWeaver” as shown below.

要启动 PathWeaver，请点击 Visual Studio Code 界面右上角的省略号图标。必须先选中 WPILib 项目中的源文件，才会看到该图标。然后点击 “Start tool”，再点击 “PathWeaver”，如下所示。



In VSCode select “...” in the upper right corner and then “Start Tool”. / 在 VSCode 右上角选择 “...”，然后选择 “Start Tool”。

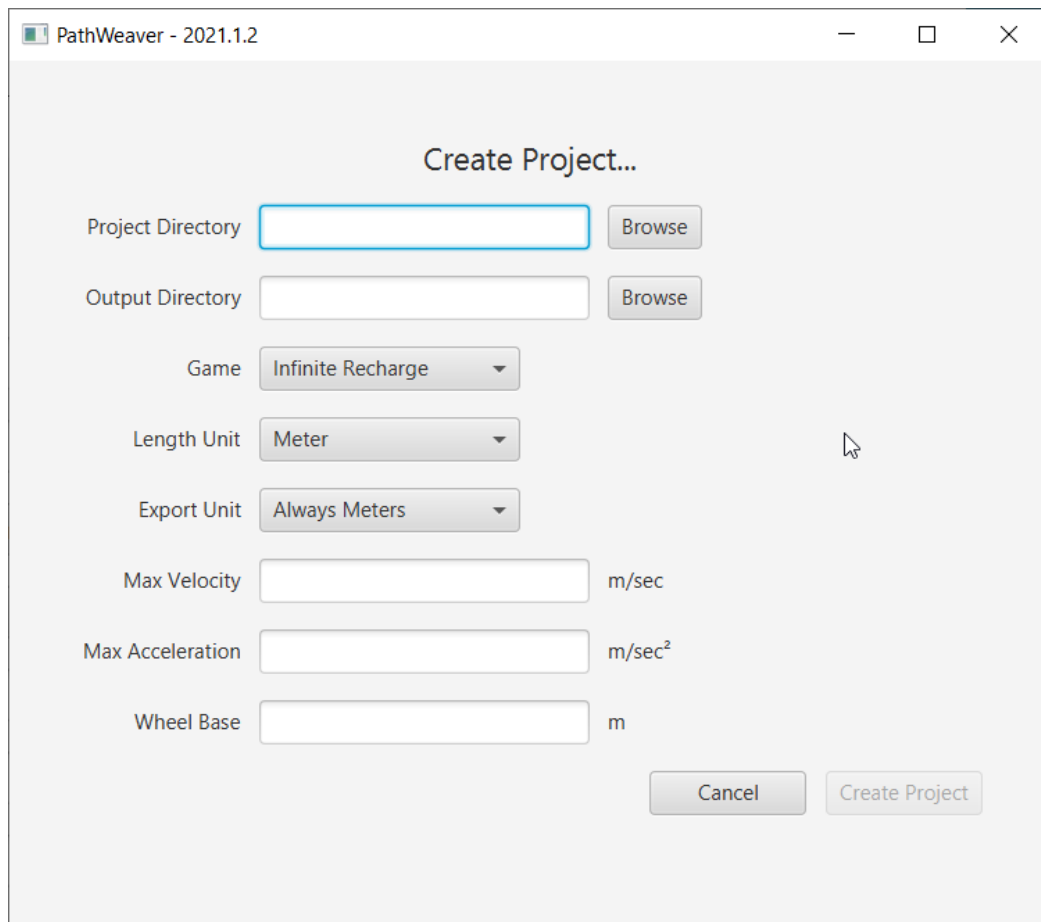


Choose PathWeaver from the tool dropdown list. / 从工具下拉列表中选择 PathWeaver。

Creating the Project / 创建项目

To create a PathWeaver project, click on “Create project” and then fill out the project creation form. Notice that hovering over any of the fields in the form will display more information about what is required.

要创建 PathWeaver 项目，请点击 “Create project”，然后填写项目创建表单。请注意，将鼠标悬停在表单的任意字段上，会显示该字段要求的更多信息。



The PathWeaver project creation screen. / PathWeaver 项目创建界面。

Project Directory: This is normally the top level project directory that contains the `build.gradle` and `src` files for your robot program. Choosing this directory is the expected way to use PathWeaver and will cause it to locate all the output files in the correct directories for automatic path deployment to your robot.

Project Directory: 通常是机器人程序的项目根目录，其中包含 `build.gradle` 和 `src` 文件。选择此目录是 PathWeaver 的预期使用方式，它会将所有输出文件放在正确目录，以便将路径自动部署到机器人。

Output directory: The directory where the paths are stored for deployment to your robot. If you specified the top level project folder of our robot project in the previous step (as recommended) filling in the output directory is optional.

Output directory: 存储路径以供部署到机器人的目录。如果上一步按建议指定了机器人项目的根文件夹，则可以不填写输出目录。

Game: The game (which FRC® game is being used) will cause the correct field image overlay to be used. You can also create your own field images and the procedure will be described later in this series.

Game: 选择使用哪一年的 FRC® 赛事，以加载正确的场地图像叠加层。你也可以创建自己的场地图像；本系列后文会介绍步骤。

Length Unit: The units to be used in describing your robot and for the field measurements when visualizing trajectories using PathWeaver.

Length Unit: 在 PathWeaver 中描述机器人以及可视化轨迹时表示场地测量值所用的单位。

Export Unit: The units to be used when exporting the paths and waypoints. If you are planning to use WPILib Trajectories, then you should choose Always Meters. Choosing Same as Project will export in the same units as Length Unit above.

Export Unit: 导出路径和路径点时使用的单位。如果计划使用 WPILib Trajectories，应选择 Always Meters。选择 Same as Project 则使用上方 Length Unit 所指定的单位导出。

Max Velocity: The max speed of the robot for trajectory tracking. The kitbot runs at ~10 ft/sec which is ~3 m/sec.

Max Velocity: 机器人跟踪轨迹时的最大速度。标准套件机器人约以 10 ft/sec，也就是约 3 m/sec 的速度行驶。

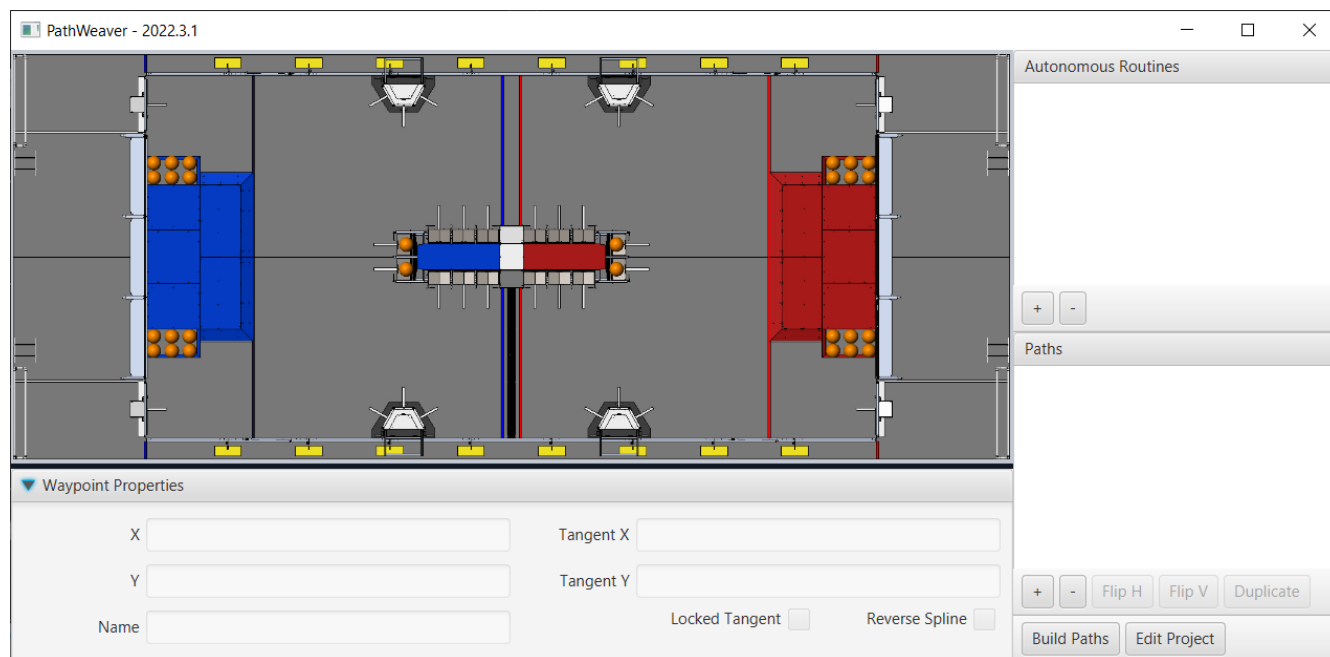
Max Acceleration: The max acceleration of the robot for trajectory tracking. Using a conservative 1 m/sec² is a good place to start if you don't know your drivetrain's characteristics.

Max Acceleration: 机器人跟踪轨迹时的最大加速度。如果了解底盘特性，可以先保守地设为 1 m/sec²。

Wheel Base: The distance between the left and right wheels of your robot. This is used to ensure that no wheel on a differential drive will go over the specified max velocity around turns.

Wheel Base: 机器人左右车轮之间的距离。该值用于确保差速驱动机器人转弯时，任一车轮都不会超过指定的最大速度。

PathWeaver User Interface / PathWeaver 用户界面



An overview of the PathWeaver user interface. / PathWeaver 用户界面概览。

The PathWeaver user interface consists of the following:

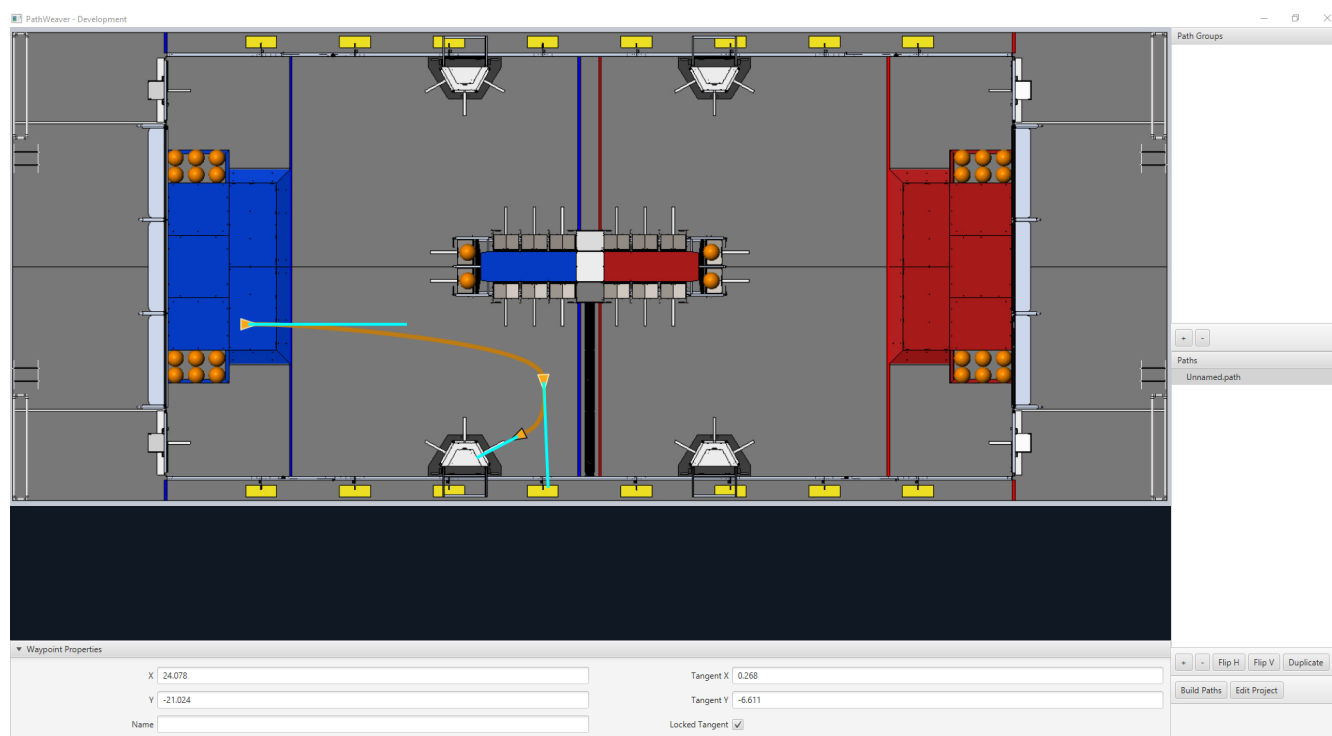
PathWeaver 用户界面由以下部分组成：

1. The field area in the top left corner, which takes up most of the PathWeaver window. Trajectories are drawn on this part of the program.
1. 左上角占据 PathWeaver 窗口大部分空间的场地区域。轨迹在程序的这一区域绘制。
2. The properties of the currently selected waypoint are displayed in the bottom pane. These properties include the X and Y, along with the tangents at each point.
2. 当前选中路径点的属性显示在底部窗格中，包括每个点的 X、Y 以及切线。
3. A group of paths (or an “autonomous” mode) is displayed on the upper right side of the window. This is a convenient way of seeing all of the trajectories in a single auto mode.
3. 一组路径（或一种“自主”模式）显示在窗口右上方。这样可以方便地查看某一自主模式中的所有轨迹。
4. The individual paths that a robot might follow are displayed in the lower right side of the window.
4. 机器人可能跟随的各条独立路径显示在窗口右下方。
5. The “Build Paths” button will export the trajectories in a JSON format. These JSON files can be used from the robot code to follow the trajectory.
5. “Build Paths” 按钮会以 JSON 格式导出轨迹。机器人代码可使用这些 JSON 文件跟随轨迹。
6. The “Edit Project” button allows you to edit the project properties.
6. “Edit Project” 按钮可用于编辑项目属性。

Visualizing PathWeaver Trajectories / 可视化 PathWeaver 轨迹

PathWeaver's primary feature is to visualize trajectories. The following images depict a smooth trajectory that represents a trajectory that a robot might take during the autonomous period. Paths can have any number of waypoints that can allow more complex driving to be described. In this case there are 3 waypoints (including the start and stop) depicted with the triangle icons. Each waypoint consists of a X, Y position on the field as well as a robot heading described as the X and Y tangent lines.

PathWeaver 的主要功能是将轨迹可视化。以下图片展示一条平滑轨迹，代表机器人在自主阶段可能行驶的路线。路径可以有任意数量的路径点，从而描述更复杂的行驶过程。本例以三角形图标标出了 3 个路径点（包括起点和终点）。每个路径点包含场地上的 X、Y 位置，以及由 X、Y 切线描述的机器人朝向。

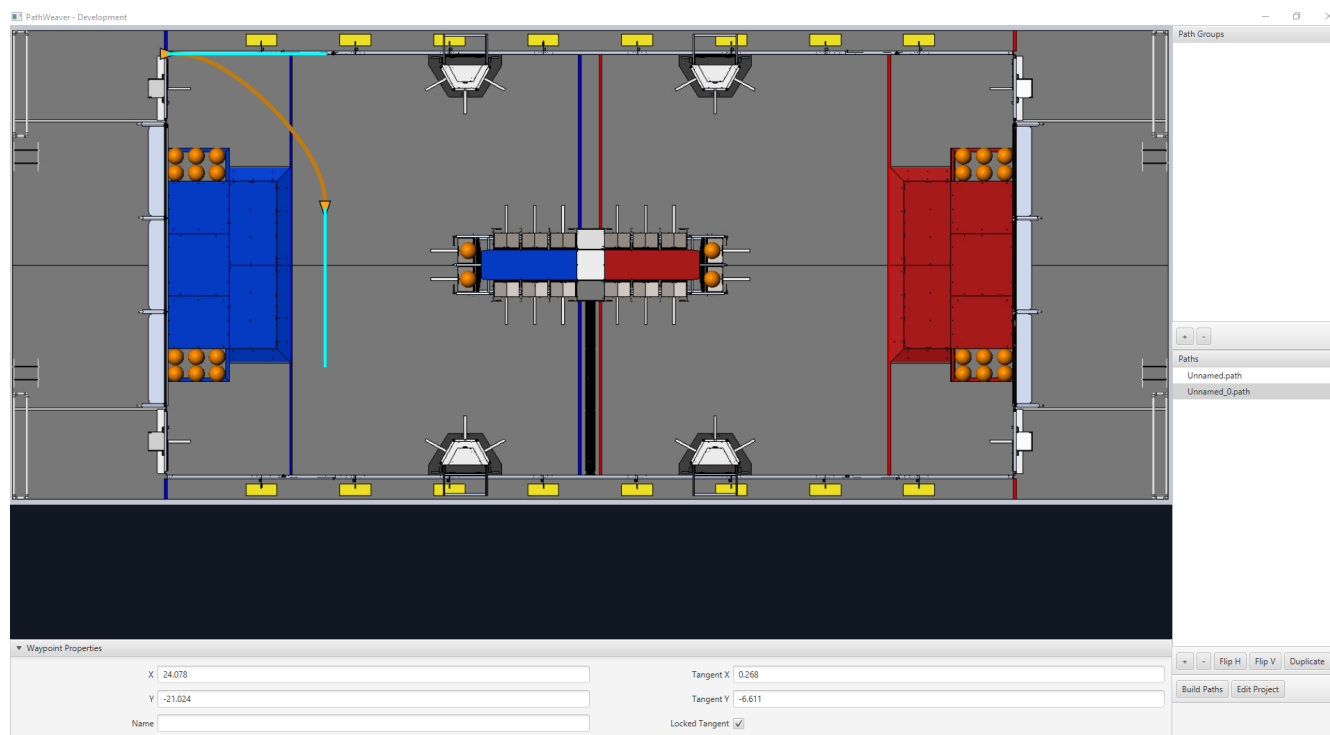


A complete PathWeaver path to the back side of the rocket in Deep Space. / 2019 Deep Space 中通往火箭背面的完整 PathWeaver 路径。

Creating the Initial Trajectory / 创建初始轨迹

To start creating a trajectory, click the + (plus) button in the path window. A default trajectory will be created that probably does not have the proper start and end points that you desire. The path also shows the tangent vectors (teal lines) for the start and end points. Changing the angle of the tangent vectors changes the shape of the trajectory.

要开始创建轨迹，请点击路径窗口中的 +（加号）按钮。系统会创建一条默认轨迹，但它的起点和终点可能不是你想要的位置。路径还会显示起点与终点的切向量（青绿色线条）。改变切向量的角度就会改变轨迹形状。



The waypoint positions when a path is first created. The first is in the far upper left corner. The second is turned 90 degrees facing downward. / 刚创建路径时的路径点位置。第一个点在最左上角；第二个点旋转了 90 度，朝向下方。

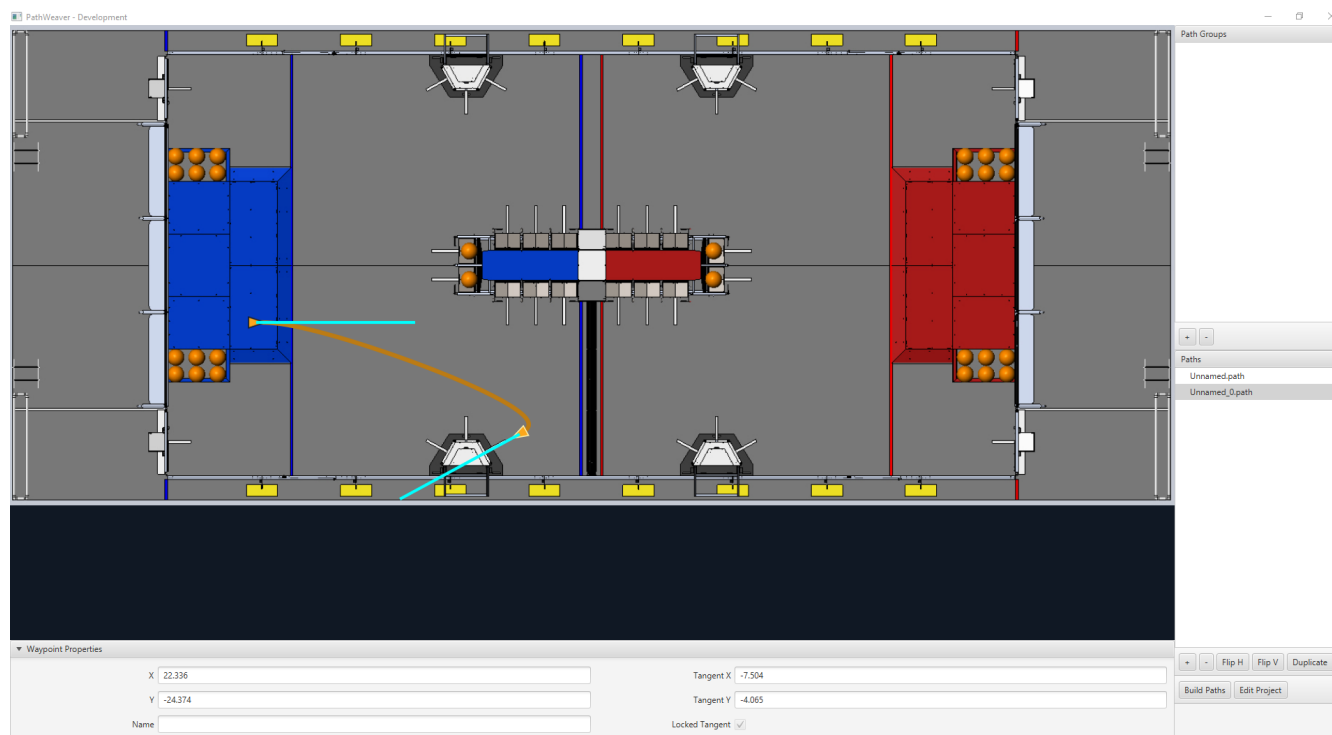
Drag the start and end points of the trajectory to the desired locations. Notice that in this case, the default trajectory does not start in a legal spot for the 2019 game. We can drag the initial waypoint to make the robot start on the HAB.

将轨迹的起点和终点拖到目标位置。请注意，在本例中，默认轨迹并未从 2019 年比赛允许的位置起步。我们可以拖动初始路径点，使机器人从 HAB 起步。

Changing a Waypoint Heading / 改变路径点朝向

The robot heading can be changed by dragging the tangent vector (teal) line. Here, the final waypoint was dragged to the desired final pose and was rotated to face the rocket.

拖动切向量（青绿色线条）可改变机器人朝向。这里将终点路径点拖到了期望的最终 pose，并旋转到朝向火箭的位置。

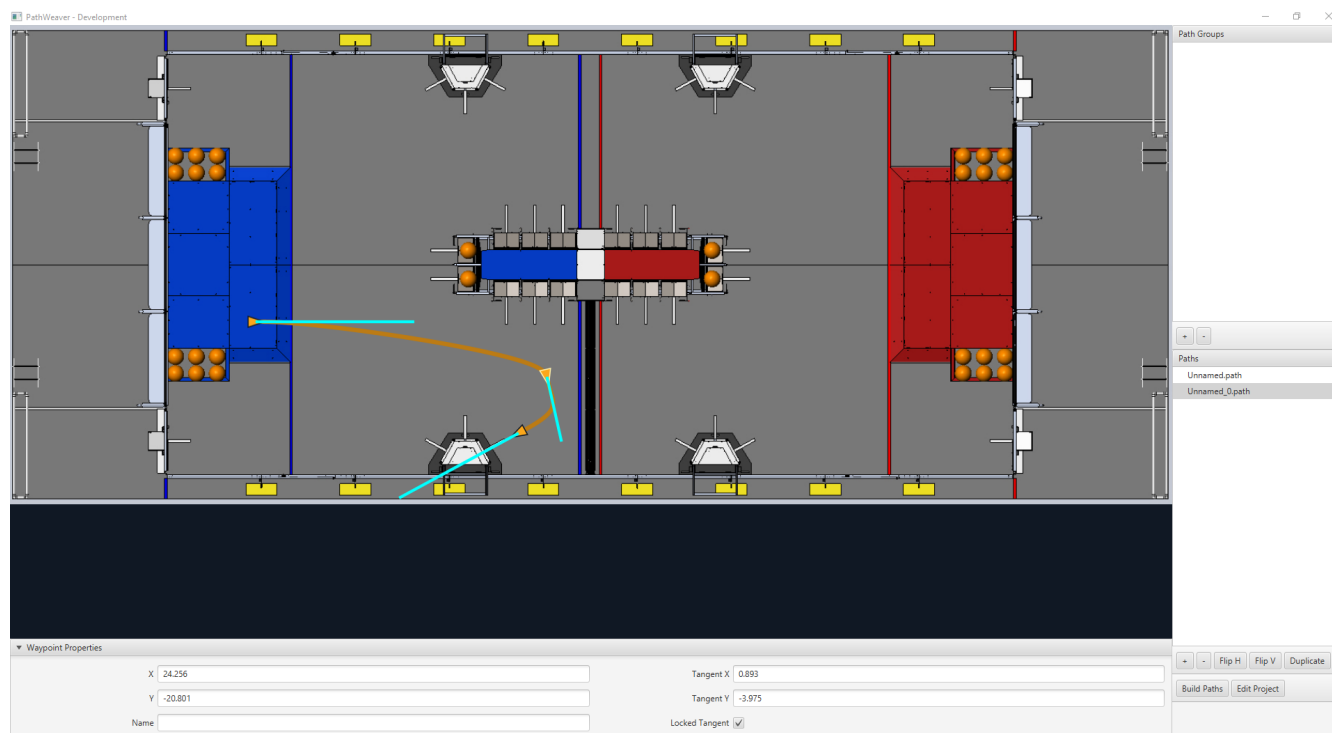


Adjusting the blue line out the front of each waypoint controls the direction it is facing. / 调整从各路径点前方伸出的蓝线，可控制该路径点所面向的方向。

Adding Additional Waypoints to Control the Robot Path / 添加更多路径点以控制机器人路径

Adding additional waypoints and changing their tangent vectors can affect the path that is followed. Additional waypoints can be added by dragging in the middle of the path. In this case, we added another waypoint in the middle of the path.

添加路径点并改变其切向量，会影响机器人跟随的路径。可以在路径中间拖动以添加路径点。本例在路径中间又添加了一个路径点。



Adding an additional waypoint by clicking and dragging on the middle of the path. / 点击并拖动路径中部，添加额外的路径点。

Locking the Tangent Lines / 锁定切线

Locking tangent lines prevents them from changing when the trajectory is being manipulated. The tangent lines will also be locked when the point is moved.

锁定切线后，操作轨迹时切线不会改变。移动该点时，切线也会保持锁定。

More Precise control of Waypoints / 更精确地控制路径点

While PathWeaver makes it simple to draw trajectories that the robot should follow, it is sometimes hard to precisely set where the waypoints should be placed. In this case, setting the waypoint locations can be done by entering the X and Y value which might come from an accurate CAD model of the field. The points can be entered in the X and Y fields when a waypoint is selected.

虽然 PathWeaver 让绘制机器人应跟随的轨迹变得简单，但有时难以精确设置路径点位置。这时可输入 X、Y 数值来设置路径点位置，这些数值可以来自准确的场地 CAD 模型。选中路径点后，可在 X、Y 字段输入坐标。

Creating Autonomous Routines / 创建自主例程

Autonomous Routines (also known as Path Groups) are a way of visualizing where one path ends and the next one starts. An example is when the robot program drives one path, does something after the path has completed, drives to another location to obtain a game piece, then back again to score it. It's important that the start and end points of each path in the routine have common end and start points. By adding all the paths to a single autonomous routine and selecting the routine, all paths in that routine will be shown. Then each path can be edited while viewing all the paths.

自主例程（也称为路径组）可用来可视化一条路径在哪里结束、下一条路径在哪里开始。例如，机器人程序先沿一条路径行驶，路径结束后执行某个动作，再前往另一位置取得场上物件，最后返回得分。在例程中，各路径的终点和下一条路径的起点必须相接。将所有路径加入同一个自主例程并选中该例程后，例程中的所有路径都会显示出来，此时可以一边查看全部路径，一边编辑各条路径。

Creating an Autonomous Routine / 创建自主例程

Press the + button underneath Autonomous Routines. Then drag the Paths from the Paths section into your Autonomous Routine.

点击 Autonomous Routines 下方的 + 按钮。然后把 Paths 区域中的路径拖入你的 Autonomous Routine。

Each path added to an autonomous routine will be drawn in a different color making it easy to figure out what the name is for each path.

加入自主例程的每条路径都会以不同颜色绘制，便于区分各条路径的名称。

If there are multiple paths in a routine, selection works as follows:

当例程包含多条路径时，选择操作的效果如下：

1. Selecting the routine displays all paths in the routine making it easy to see the relationship between them. Any waypoint on any of the paths can be edited while the routine is selected and it will only change the path containing the waypoint.
1. 选中整个例程时，会显示例程中的所有路径，便于查看它们之间的关系。选中例程期间，可以编辑任一路径上的任一路径点，且只会改变该路径点所属的路径。
2. Selecting on a single path in the routine will only display that path, making it easy to precisely see what all the waypoints are doing and preventing clutter in the interface if multiple paths cross over or are close to each other.
2. 选中例程中的单条路径时，只显示该路径，便于准确观察各路径点的作用；如果多条路径交叉或彼此接近，这样也能避免界面杂乱。

Importing a PathWeaver JSON / 导入 PathWeaver JSON

The `TrajectoryUtil` class can be used to import a PathWeaver JSON into robot code to follow it. This article will go over importing the trajectory. Please visit the [end-to-end trajectory tutorial](#) for more information on following the trajectory.

可以使用 `TrajectoryUtil` 类将 PathWeaver JSON 导入机器人代码，以便跟随该轨迹。本文将介绍如何导入轨迹。有关跟随轨迹的更多信息，请参阅[端到端轨迹教程](#)。

The `fromPathweaverJson` (Java) static method in `TrajectoryUtil` can be used to create a trajectory from a JSON file stored on the roboRIO file system.

`TrajectoryUtil` 中的 `fromPathweaverJson` (Java) 静态方法可从存储在 roboRIO 文件系统中的 JSON 文件创建轨迹。

Important / 重要

To be compatible with the Field2d view in the simulator GUI, the coordinates for the exported JSON have changed. Previously (before 2021), the range of the y-coordinate was from -27 feet to 0 feet whereas now, the range of the y-coordinate is from 0 feet to 27 feet (with 0 being at the bottom of the screen and 27 feet being at the top). This should not affect teams who are correctly [resetting their odometry to the starting pose of the trajectory](#) before path following.

为了兼容仿真器 GUI 中的 Field2d 视图，导出的 JSON 坐标已发生变化。以前（2021 年之前），y 坐标范围为 -27 英尺至 0 英尺；现在范围为 0 英尺至 27 英尺（屏幕底部为 0，顶部为 27 英尺）。对于在跟随路径前正确将 [odometry 重置到轨迹起始 pose](#) 的队伍，这不应造成影响。

Note / 说明

PathWeaver places JSON files in `src/main/deploy/paths` which will automatically be placed on the roboRIO file system in `/home/lvuser/deploy/paths` and can be accessed using `getDeployDirectory` as shown below.

PathWeaver 将 JSON 文件放在 `src/main/deploy/paths` 中，部署时它们会自动出现在 roboRIO 文件系统的 `/home/lvuser/deploy/paths` 中，可按下文所示使用 `getDeployDirectory` 访问。

```
String trajectoryJSON = "paths/YourPath.wpilib.json";
Trajectory trajectory = new Trajectory();

public Robot() {
    try {
        Path trajectoryPath = Filesystem.getDeployDirectory().toPath().resolve(trajectoryJSON);
        trajectory = TrajectoryUtil.fromPathweaverJson(trajectoryPath);
    } catch (IOException ex) {
        DriverStation.reportError("Unable to open trajectory: " + trajectoryJSON, ex.getStackTrace());
    }
}
```

In the examples above, `YourPath` should be replaced with the name of your path.

在上述示例中，应将 `YourPath` 替换为你的路径名称。

Warning / 警告

Loading a PathWeaver JSON from file in Java can take more than one loop iteration so it is highly recommended that the robot load these paths on startup.

在 Java 中从文件加载 PathWeaver JSON 可能耗时超过一个循环周期，因此强烈建议机器人在启动时加载这些路径。

Adding field images to PathWeaver / 向 PathWeaver 添加场地图像

Here are instructions for adding your own field image using the 2019 game as an example.

以下以 2019 年比赛为例，说明如何添加自己的场地图像。

Games are loaded from the `~/PathWeaver/Games` on Linux and macOS or `%USERPROFILE%/PathWeaver/Games` directory on Windows. The files can be in either a game-specific subdirectory, or in a zip file in the Games directory. The ZIP file must follow the same layout as a game directory; the JSON file must be in the root of the ZIP file (cannot be in a subdirectory).

在 Linux 和 macOS 上，游戏配置从 `~/PathWeaver/Games` 加载；在 Windows 上，则从 `%USERPROFILE%/PathWeaver/Games` 目录加载。文件可以放在某一游戏专用子目录中，也可以放在 Games 目录中的 zip 文件里。ZIP 文件必须采用与游戏目录相同的布局；JSON 文件必须位于 ZIP 文件的根目录，不能放在子目录中。

Download the example FIRST Destination Deep Space field definition [here](#). Other field definitions are available in the [allwpilib GitHub repository](#).

可从[此处](#)下载示例 FIRST Destination Deep Space 场地定义。其他场地定义可在 [allwpilib GitHub 仓库](#)中找到。

File Layout / 文件布局

```
~/PathWeaver
  /Games
    /Custom Game
      custom-game.json
      field-image.png
    OtherGame.zip
```

JSON Format / JSON 格式

```
{
  "game": "game name",
  "field-image": "relative/path/to/img.png",
  "field-corners": {
    "top-left": [x, y],
    "bottom-right": [x, y]
  },
  "field-size": [width, length],
  "field-unit": "unit name"
}
```

The path to the field image is relative to the JSON file. For simplicity, the image file should be in the same directory as the JSON file.

场地图像路径相对于 JSON 文件。为简单起见，图像文件应与 JSON 文件位于同一目录。

The field corners are the X and Y coordinates of the top-left and bottom-right pixels defining the rectangular boundary of the playable area in the field image. Non-rectangular playing areas are not supported.

场地边界是定义场地图像中可比赛区域矩形边界的左上角和右下角像素的 X、Y 坐标。不支持非矩形的比赛区域。

The field size is the width and length of the playable area of the field in the provided units.

场地尺寸是在所给单位下可比赛区域的宽度和长度。

The field units are case-insensitive and can be in meters, cm, mm, inches, feet, yards, or miles.

Singular, plural, and abbreviations are supported (e.g. “meter”, “meters”, and “m” are all valid for specifying meters)

场地单位不区分大小写，可以使用 meters、cm、mm、inches、feet、yards 或 miles。单数、复数和缩写形式都受支持（例如，“meter” “meters” 和 “m” 都可以表示米）。

Note / 说明

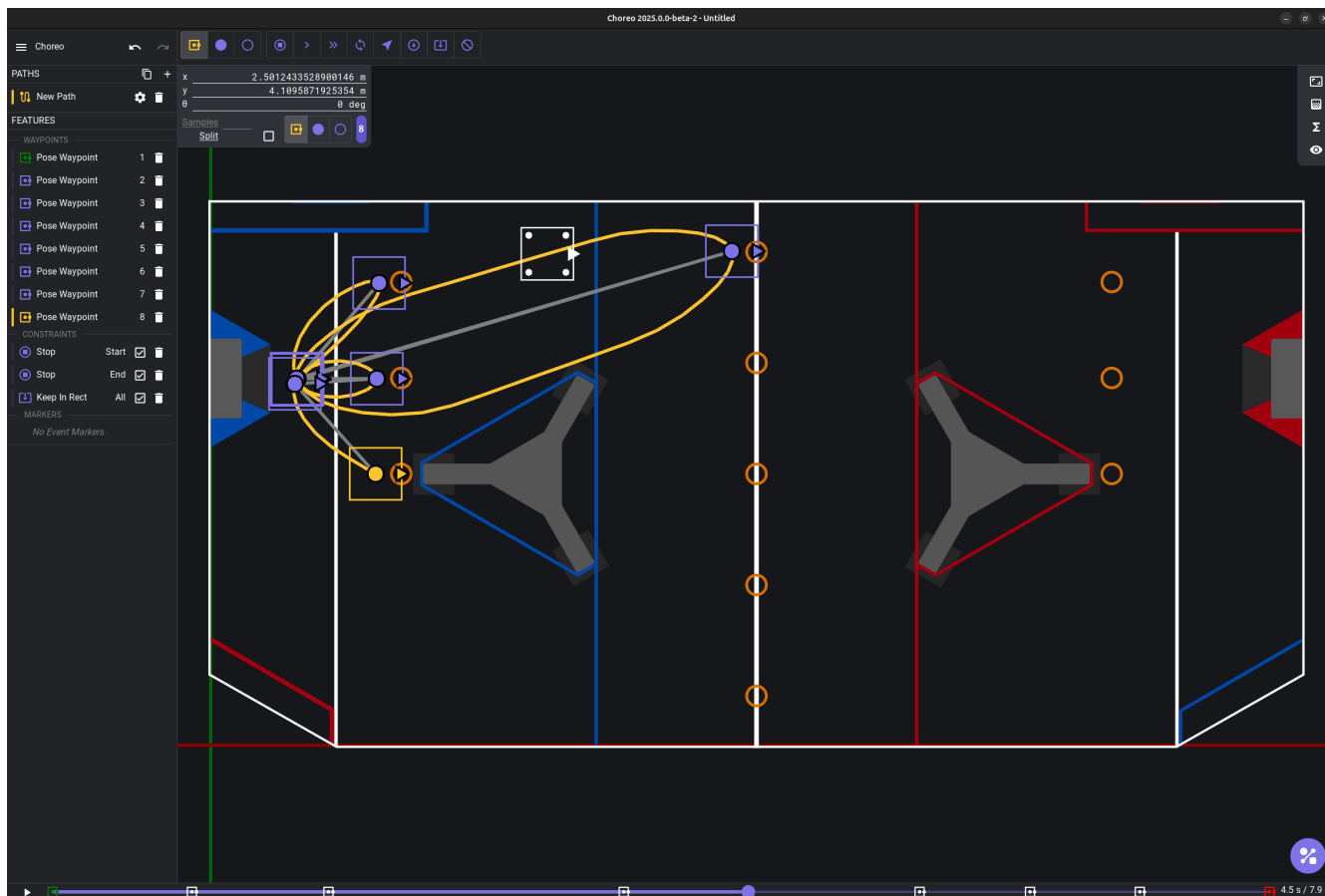
When making a new field image, a border (minimum of 20 pixels is recommended) should be left around the outside so that waypoints on the field edge are accessible.

制作新场地图像时，外缘应留出边框（建议至少 20 像素），以便访问位于场地边缘的路径点。

Choreo / Choreo

Choreo is a tool for calculating time optimal autonomous trajectories. This allows simple and efficient creation of autonomous routines.

Choreo 是计算时间最优自主轨迹的工具。它使创建自主例程变得简单而高效。



Screenshot of Choreo showing the 2024 FRC field with a trajectory plotted on it / Choreo 截图：2024 年 FRC 场地上绘有一条轨迹。

Note / 说明

Detailed documentation for Choreo can be found [here](#).

Choreo 的详细文档可在[此处](#)查看。

The capabilities of Choreo include

Choreo 的功能包括：

- creating trajectories with a variety of different constraints
- 在各种不同约束下创建轨迹
- scheduling commands at different events on your path
- 在路径上的不同事件处调度 Command
- support for different drivebases, including swerve and tank
- 支持包括 swerve 和 tank 在内的不同底盘类型